研究微网与外部电网电力调控策略问题

摘要

随着"双碳"战略的深入推进，以分布式光伏和电池储能为核心的小型微网快速发展。微网通过与外部电网的互动调控，在低价时段购电蓄能、在高价时段放电供负荷，既能提升新能源就地消纳水平，又能显著降低用电成本。如何根据负荷、光伏与储能状态精准制定购电与充放电策略，在满足供电可靠性的前提下实现经济性最优，是微网运行中的核心问题。本文围绕这一问题，构建了四个层层递进的优化调度模型。

针对问题 1：在电价与负荷恒定、光伏出力已知的确定条件下，以 10 分钟为调度时步，构建了以全天购电费用最小为目标的日前调度线性规划模型。模型综合考量供需平衡、储能荷电状态转移、充放电功率限值以及 0:00 与 24:00 储电量闭环边界等约束，调用 HiGHS 求解器全局求解。结果表明：全天购电量为 59482.70 kWh，购电费为 35126.95 元；光伏大发时段外网购电压至 0，实现新能源全额消纳；储能执行"两充两放"峰谷套利策略，荷电状态始终运行在[1200, 10800] kWh 安全区间内。

针对问题 2：针对负荷与光伏随机波动且缺电需承担 5 倍惩罚的场景，提出了"日前鲁棒计划+日内 EMS 状态机执行"的两阶段滚动优化框架。日前以近邻 7 日历史加权均值预测为基准，引入负荷上偏、光伏下折的保守置信因子，并设置 300 kWh 防穿缓冲以抵御高额违约风险；日内由能量管理系统按"富余优先充电、缺额优先放电、储能极限后紧急购电兜底"的物理优先级执行。全年 365 天滚动仿真表明：6 月 21 日光伏充裕，实现零紧急购电；而 12 月 21 日因日照时长短、采暖负荷高企，单日紧急购电超 2 万 kWh，揭示了微网保供压力的显著季节差异。

针对问题 3：引入 0:00、6:00、12:00、18:00 四次光伏预报更新机制，构建了日前基准层、日内滚动跟踪层与实时 EMS 执行层相嵌套的三阶段模型预测控制（MPC）架构，并通过引入正、负偏差变量将多级非对称结算规则线性化，使各层均保持线性规划结构、可高效求解。结果表明：日内动态纠偏使 12 月 21 日紧急购电量由 20647 kWh 大幅压缩至 6247 kWh，降幅达 69.7%；在个别日光以局部越限为代价换取全局最优，全年综合用电成本下降超 40%。进一步从相变边界覆盖、储能硬件滤波效应与边际效益帕累托递减三方面论证了现有四次预报已接近信息增益上限，无需引入更多预报时刻。

针对问题 4：将微网内部物理约束与波动电价下的多级结算机制相统一，构建了兼容实时电价场景的通用时序优化模型，通过开关日内调整通道即可退化求解问题 2 与问题 3 两种运行模式。结果表明：在电价尖峰脉冲下，储能充放电拐点由"时钟驱动"转变为"价差驱动"；MPC 模式利用 1.5 倍边际低价提前增购、置换 5 倍紧急惩罚，使 12 月 21 日电费降低 8.4%、9 月 23 日降低 16.3%，充分展现了日内动态纠偏机制平抑现货市场风险的经济价值。

最后，我们对提出的模型进行全面的评价：本文的模型贴合实际，能合理解决提出的问题，具有实用性强、算法效率高等特点。

关键词：微网优化调度；峰谷套利；模型预测控制；鲁棒优化；线性规划。

一、问题重述

问题 1：电价与负荷恒定，给定一天光伏发电预测数据，要求微网供电不低于负荷，且储能在 0:00 与 24:00 电量相同，建立每天 0:00 的计划购电策略模型并给出全天最优调度方案。

问题 2：电价恒定，负荷与光伏随时间变化。微网供电不足时需以 5 倍电价紧急购电，其余时段按计划购电量计费。建立每天 0:00 的计划购电策略模型，并基于全年实际数据滚动仿真，统计指定日期的购电、充放电与紧急购电结果。

问题 3：在每天 0:00、6:00、12:00、18:00 可获得未来 24 小时整点光伏预报。计划购电量高于调整量部分按 50%电价退还，调整量高于计划量部分按 150%电价结算，另有 5 倍紧急购电惩罚。建立含调整机制的购电策略模型，并分析是否需要引入其他预报时刻。

问题 4：外网电价实时波动。针对波动电价建立通用购电策略模型，并基于波动电价数据重新求解问题 2 与问题 3。

二、问题分析

2.1 问题 1 分析

问题 1 属于确定条件下的静态优化问题。电价、负荷已知，光伏出力为确定性预测值，决策变量仅为各时段购电量与储能充放电量。关键约束有三：一是各时段供电量不低于负荷（允许弃光，不允许缺电）；二是储能状态随充放电按效率递推，并始终处于[1200, 10800] kWh 安全区间；三是 0:00 与 24:00 储电量相等构成日循环闭环。目标函数为全天购电费用最小，决策变量均为连续变量，目标与约束均为线性，故建立标准线性规划模型，采用单纯形法即可求得全局最优解。问题的核心在于利用峰谷电价差安排储能"低充高放"，并利用正午光伏盈余为储能充电[1]。

2.2 问题 2 分析

问题 2 引入了两类不确定性：负荷与光伏的日内随机波动，以及 5 倍缺电惩罚的高风险。由于计划购电量在 0:00 申报后不可调整，日前决策必须兼顾经济性与鲁棒性[2]。为此采用"日前防御+日内执行"的两阶段框架：日前基于历史近邻数据加权预测，施加负荷上偏、光伏下折的保守因子并预留防穿缓冲，以较小成本防御大幅惩罚；日内由 EMS 状态机执行——电量富余时优先充电，缺额时优先放电，储能达功率或容量极限后的残余缺口触发紧急购电。需要全年逐日滚动仿真，通过跨日状态继承体现储能运行的真实连续性。

2.3 问题 3 分析

问题 3 将问题 2 的"静态防御"转变为"日内动态纠偏"。由于增量调整电价（1.5 倍）远低

于紧急购电价（5 倍），且减量可获 50%返还，日前不必过度保守，而应在预报更新时刻滚动修正剩余时段计划。这契合模型预测控制（MPC）思想：以 0:00 预报制定基准计划，在 6:00、12:00、18:00 以实测储电量为初始状态、以最新预报为输入滚动重优化剩余时段。难点在于非对称的多级结算规则，可通过引入正、负偏差变量将其线性化，使每层均保持线性规划结构。此外需从信息价值角度论证预报时刻的取舍。

2.4 问题 4 分析

问题 4 中电价由固定阶梯变为实时波动，微网内部物理约束（供需平衡、储能状态转移、功率限值、EMS 响应逻辑）均不变，变化的是价格信号与结算时点。因此可将问题 2、3 的模型统一为计及多级价格结算与跨日状态延续的通用时序优化模型，通过关闭或开启日内调整通道退化求解两种模式。分析重点转向：在电价尖峰脉冲下，储能充放电拐点由"时钟驱动"转为"价差驱动"，MPC 模式能否利用日内低价窗口提前增购、置换高额紧急惩罚，从而量化动态纠偏机制对现货市场风险的平抑效果。

三、模型假设

为确保数学模型的严谨性、可计算性并贴合工程实际，提出以下合理假设：

1. **步内稳态假设**：取 Δt=10 min 为离散调度时步，步内电价、出力与负荷恒定[3]。
2. **储能效率常数假设**：储能充放电效率恒为 90%，忽略温度、荷电状态与倍率引起的效率波动。
3. **微网内部损耗忽略假设**：微网内部线路与变流器损耗远小于传输功率，模型中不计内部网损。
4. **单向功率流动与盈余弃电假设**：微网不向外部电网反送电（购电量非负）；储能充满后仍富余的光伏电能无法消纳，按弃电处理。
5. **电网容量充裕假设**：外网容量无限，任何时刻的计划购电与紧急购电均可被满足，不受上级变压器容量约束。

四、符号说明

符号	单位	物理意义与说明	属性
T	-	全天日前调度的离散时步总数	常数
Δt	h	单个离散调度时间步长	常数
t	-	时间离散步长索引	索引
c_t	元/kWh	调度时段 t 对应的外部电网销售电价	已知参数
$P_{\text{load},t}$	kW	调度时段 t 的小区负荷平均预测功率	已知参数
$P_{\text{pv},t}$	kW	调度时段 t 的分布式光伏发电平均预测功率	已知参数
$P_{\max}$	kW	储能设备允许的最大瞬时充、放电功率	系统参数
C_{cap}	kWh	储能设备的额定标称能量容量	系统参数
$S_{\min},S_{\max}$	kWh	储能设备运行允许的储电量下限与上限	系统参数
S_0,S_T	kWh	储能设备在 0:00 与 24:00 规定的储电量基准	边界参数
$\eta_{\text{ch}},\eta_{\text{dis}}$	-	储能系统充电效率与放电效率	物理常数
$P_{\text{grid},t}$	kW	调度时段 t 微网向外部电网购买的有功功率	决策变量
$P_{\text{ch},t}$	kW	调度时段 t 储能设备吸收的充电功率	决策变量
$P_{\text{dis},t}$	kW	调度时段 t 储能设备输出的放电功率	决策变量
S_t	kWh	调度时段 t 结束时储能设备内部的实际储电量	状态变量
$E_{\text{grid},t}$	kWh	调度时段 t 内微网向外网购买的电能	导出变量
$E_{\text{ch},t}$	kWh	调度时段 t 内微网向储能充入的电能	导出变量
$E_{\text{dis},t}$	kWh	调度时段 t 内储能向微网释放的电能	导出变量

五、模型的建立与求解

5.1 问题 1 模型的建立与求解

5.1.1 目标函数构建

微网调控的根本经济目标是充分利用峰谷电价差以及光伏自发自用优势，制定最优的购电时序，使得全天向主电网购电的总经济成本最低。因此，优化目标函数定义为全天各时段购电费用之和：

$$\min Z=\sum_{t=1}^{T} c_t \cdot P_{\mathrm{grid},t} \cdot \Delta t=\sum_{t=1}^{T} c_t \cdot E_{\mathrm{grid},t} \tag{1}$$

5.1.2 约束条件

系统的优化运行严格受制于电能供需平衡、储能荷电动态演化、充放电倍率约束以及日运行闭环边界条件：

1. 微网供需电能平衡与负荷保障约束

微网提供的综合电能不可低于小区负荷。每个时段的供给来自外网购电、光伏出力和储能放电，去向是小区负荷和储能充电，即：

$$-E_{\mathrm{grid},t}+E_{\mathrm{ch},t}-E_{\mathrm{dis},t} \le P_{\mathrm{pv},t}\Delta t-P_{\mathrm{load},t}\Delta t, \quad \forall t \in \{1,2,\ldots,T\} \tag{2}$$

2. 储能动态平衡与状态转移方程

储能系统的内部荷电量随充放电行为连续变化。考虑电池充放电效率的物理过程：充电时只有乘以效率的部分真正进入电池，放电时电池内部要消耗更多：

$$S_t=S_{t\text{-}1}+\left(\eta_{\mathrm{ch}} P_{\mathrm{ch},t}-\frac{1}{\eta_{\mathrm{dis}}} P_{\mathrm{dis},t}\right)\Delta t, \quad \forall t \in \{1,2,\ldots,T\}, S_0=S_T=6000\ \mathrm{kWh} \tag{3}$$

3. 储能容量安全运行边界

为防止电池过度充放电造成电化学衰减和安全隐患，运行过程中任意时刻的储电量锁定在安全窗口之内：$1200\ \mathrm{kWh} \le S_t \le 10800\ \mathrm{kWh}$。

4. 储能设备充放电功率上下限约束

储能系统变流器受额定功率限制，单时步内的充放电量不得超出最大功率约束：

$$0 \le E_{ch,t}, E_{dis,t} \le P_{max}\Delta t \tag{4}$$

5. 充放电互斥约束

物理上储能不可能同时充放电，因此有$E_{\mathrm{ch},t} \cdot E_{\mathrm{dis},t}=0$。

5.1.4 求解算法与数值实现

本模型决策变量均为连续型变量，目标函数与约束条件均关于变量线性，属于标准的大规模连续线性规划。全天 144 个调度步共计 576 个决策变量与 433 个约束。本文调用 Python 计算库中的 HiGHS 优化求解器并基于对偶单纯形法进行矩阵求解，解向量严格满足物理互斥与日闭环边界。

5.1.5 表格计算结果输出

根据求解所得的最优调度决策向量，完整策略数据已写入 result1.xlsx，并按照题目要求的格式整理出微网在各关键时段的数据表格：

表 1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0	12:00-12:10	480.4124	14:00-14:10	0
16:00-16:10	445.4317	18:00-18:10	531.8940	20:00-20:10	0
全天购电量		59482.6990	全天购电费		35126.9486

表 2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4500.0000	0	4:00-8:00	833.3333	6365.8412
8:00-12:00	4787.9643	1702.9970	12:00-16:00	5286.0352	91.1014
16:00-20:00	0	5780.1319	20:00-24:00	5333.3333	2859.8681
0:00 储电量		6000.0000	24:00 储电量		6000.0000

5.1.6 图表分析与评价

为了研究日前优化调度的经济机理与物理协调关系，将优化输出的时序结果绘制为对比图并进行深度分析：

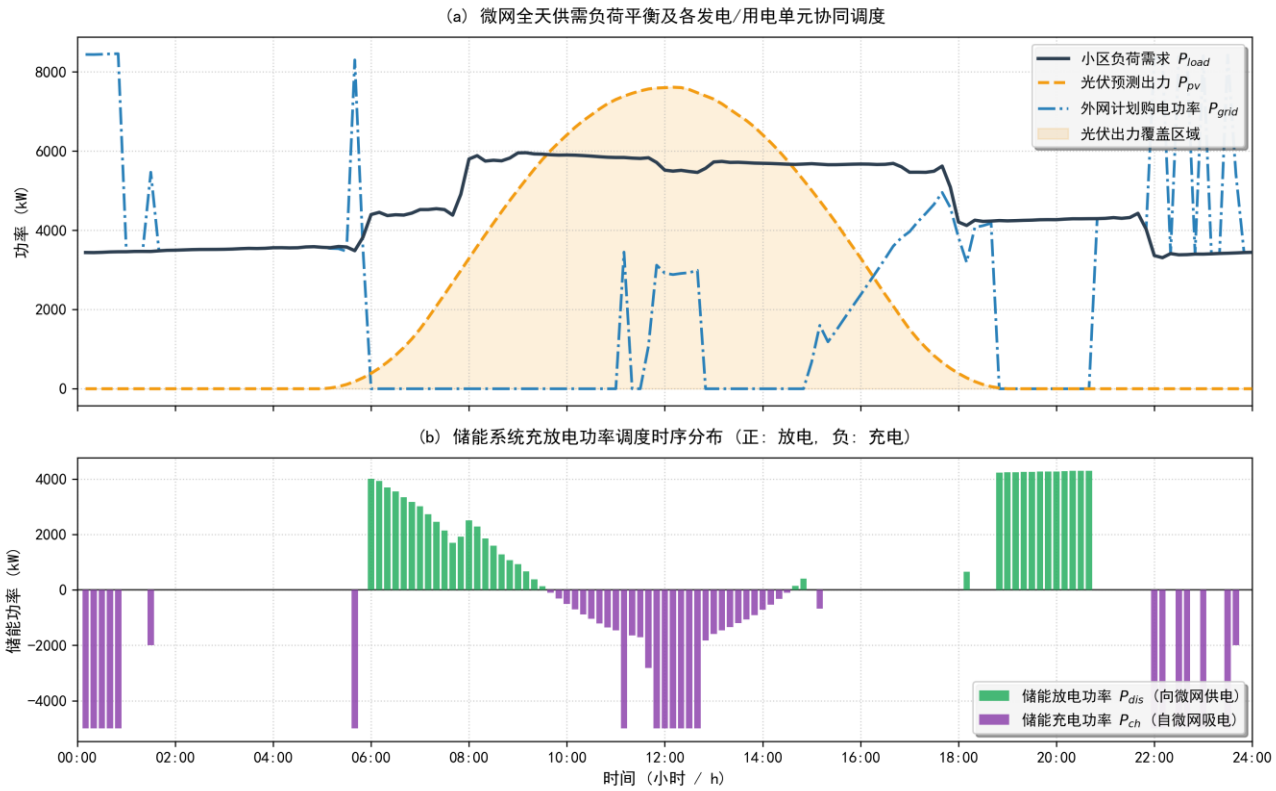


图 1 微网全天多能协同日前调度与功率平衡全景图

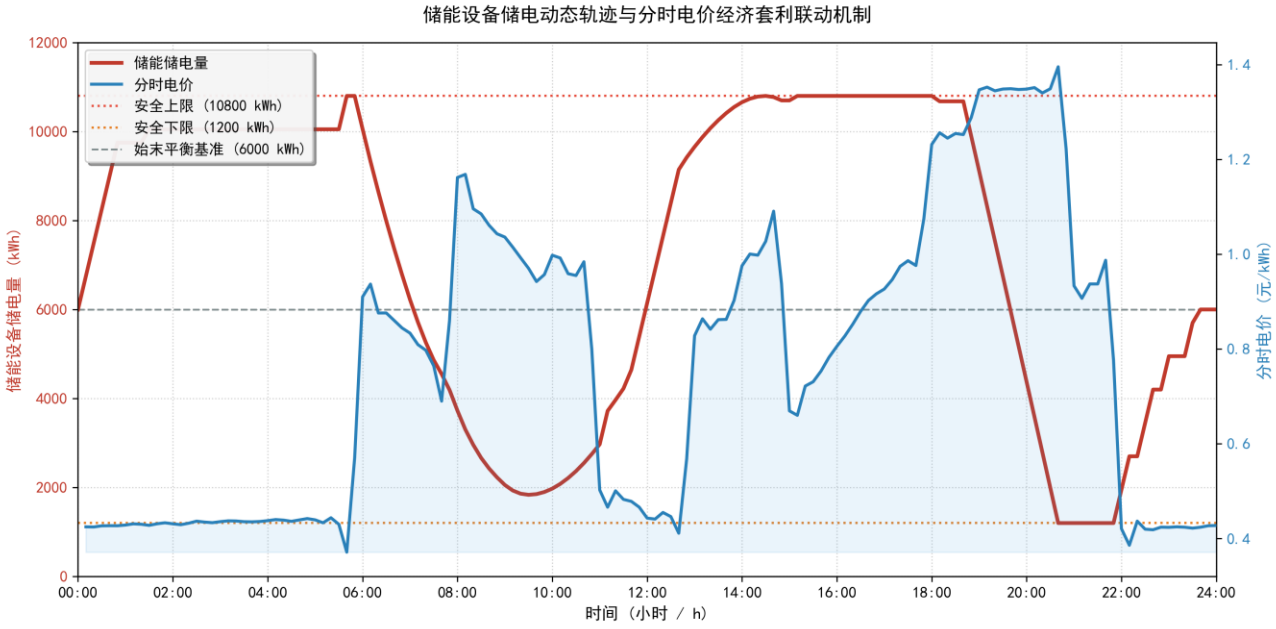


图 2 储能荷电状态动态轨迹与分时电价套利联动机制图

图 1 与图 2 表明，日前策略呈现出高度协调的物理自洽性：

1. **新能源全额消纳**：10:00–15:00 光伏大发期，外网购电压降至 0，储能以 5000 kW 满

功率吸纳盈余光伏，日间弃光率为 0；

2. **两充两放峰谷套利**：储能在凌晨谷电价期与正午光伏过剩期分别完成充能，并在早峰与晚尖峰期顶格放电保供，严格在 [1200,10800] kWh 安全区间内运行，并在 24:00 精准回归 6000 kWh 基准，实现跨日可持续循环。

5.2 问题 2 模型的建立与求解

5.2.1 日前滚动预测与不确定性置信防御建模

微网在每日 0:00 需对未来 24 小时进行滚动预测。以 2025 年 1 月 1 日至 1 月 31 日共 31 天的数据作为冷启动样本池，构建基于历史加权与时序趋势回归的在线预测模型。

日前以近邻 7 日历史加权均值作为基准预测序列 $\hat{P}_{load,t}$ 与 $\hat{P}_{pv,t}$。为防御 5 倍缺电惩罚的高阶风险，引入保守置信因子（β_{load}=1.03,β_{pv}=0.95），形成日前鲁棒规划的输入电量序列[4]：

$$E_{load,t}=\beta_{load}\hat{P}_{load,t}\Delta t \tag{5}$$

$$E_{pv,t}=\beta_{pv}\hat{P}_{pv,t}\Delta t \tag{6}$$

其中取 β_{load}=1.03，β_{pv}=0.95。即对负荷分布施加单侧上尾防御，同时对光伏出力进行保守下折，使日前计划具备内生性抗扰能力。

5.2.2 日前两阶段计划购电优化模型（0:00 决策）

1. 目标函数构建

在每天 0:00，微网以最小化全天日前申报购电成本为主体目标，并引入日末荷电状态软激励项：

$$\min \quad Z_{\text{day}}=\sum_{t=1}^{T} c_t E_{\text{grid},t}^{\text{plan}}-\gamma_{\text{end}}\cdot S_T^{\text{plan}} \tag{7}$$

式中，γ_{end}=0.01 元/kWh 为状态保全引导系数，防止模型因单日贪心过度消耗日末储电量，从而保障跨日连续运行的安全性。

2. 物理运行约束体系

(1) **日前预测供需平衡覆盖约束：**

$$E_{\mathrm{grid},t}^{\mathrm{plan}}+\tilde{E}_{\mathrm{pv},t}+E_{\mathrm{dis},t}^{\mathrm{plan}}-E_{\mathrm{ch},t}^{\mathrm{plan}}\geq\tilde{E}_{\mathrm{load},t},\quad \forall t\in[1,T] \tag{8}$$

(2) **储能日前状态动态演化方程：**

$$S_t^{\mathrm{plan}}=S_{t\text{-}1}^{\mathrm{plan}}+\eta_{\mathrm{ch}}E_{\mathrm{ch},t}^{\mathrm{plan}}-\frac{1}{\eta_{\mathrm{dis}}}E_{\mathrm{dis},t}^{\mathrm{plan}},\quad \forall t\in[1,T] \tag{9}$$

边界条件严格继承前一日 24:00 的真实状态：$S_0^{\mathrm{plan}}=S_T^{d\text{-}1,\mathrm{real}}$。

(3) **储能安全容量与防穿缓冲边界：**

$$S_{\min}+S_{\mathrm{buffer}}\leq S_t^{\mathrm{plan}}\leq S_{\max},\quad \forall t\in[1,T] \tag{10}$$

为避免日内实际负荷突增直接击穿下限，日前规划设定保留防御死区 S_{buffer}=300 kWh。

(4) **变流器最大充放电倍率约束：**

$$0\leq E_{\mathrm{ch},t}^{\mathrm{plan}}\leq P_{\max}\Delta t=833.3333\ \mathrm{kWh},\quad \forall t\in[1,T] \tag{11}$$

$$0\leq E_{\mathrm{dis},t}^{\mathrm{plan}}\leq P_{\max}\Delta t=833.3333\ \mathrm{kWh},\quad \forall t\in[1,T] \tag{12}$$

(5) **电网购电单向非负约束：**

$$E_{\mathrm{grid},t}^{\mathrm{plan}}\geq 0,\quad \forall t\in[1,T] \tag{13}$$

该模型为标准的连续线性规划，可将计划购电向量 $\boldsymbol{E}_{\mathrm{grid}}^{\mathrm{plan}}$在秒级内收敛至全局最优。

5.2.3 日内实时能量管理（EMS）状态机与紧急购电触发机制

日前计划购电量一旦在 0:00 申报即被固化为不可调整的申报值。进入日内执行阶段，真实负荷电量 $E_{\mathrm{load},t}^{\mathrm{real}}=P_{\mathrm{load},t}^{\mathrm{real}}\Delta t$ 与真实光伏出力 $E_{\mathrm{pv},t}^{\mathrm{real}}=P_{\mathrm{pv},t}^{\mathrm{real}}\Delta t$ 实时显现[5]。

定义调度时段 t 的**实时净能量供求缺口**为：

$$\Delta E_{\mathrm{gap},t}=E_{\mathrm{load},t}^{\mathrm{real}}-\left(E_{\mathrm{grid},t}^{\mathrm{plan}}+E_{\mathrm{pv},t}^{\mathrm{real}}\right) \tag{14}$$

系统运行遵循以下严格的物理优先级状态机控制律：

1. **当$\Delta E_{gap,t}\le 0$（电量富余）**：优先为储能充电，充电量为：

$$E_{\mathrm{ch},t}^{real}=\min\left\{-\Delta E_{gap,t},P_{max}\Delta t,\left(S_{max}-S_{t-1}^{real}\right)/\eta_{\mathrm{ch}}\right\} \tag{15}$$

溢出电量弃置，无紧急购电（$E_{em,t}=0$）。

2. **当$\Delta E_{gap,t}>0$（电量缺额）**：优先调用储能放电，放电量为：

$$E_{dis,t}^{real}=\min\left\{\Delta E_{gap,t},P_{max}\Delta t,\left(S_{t-1}^{real}-S_{min}\right)\eta_{dis}\right\} \tag{16}$$

储能达到功率上限或触碰 S_{min} 红线后的残余缺口，被动触发外网紧急购电：

$$E_{em,t}=\Delta E_{gap,t}-E_{dis,t}^{real} \tag{17}$$

全天微网综合总费用核算为：

$$Z_{total}=\sum_{t=1}^{T}\left(c_tE_{grid,t}^{plan}+5c_tE_{em,t}\right) \tag{18}$$

5.2.4 全年滚动仿真求解流程

算法在 2025 年 1 月 1 日至 12 月 31 日时域内实施逐日滚动演进。将 1 月份作为模型冷启动与平稳过渡期。将 2025 年 2 月 1 日至 12 月 31 日的完整逐日逐时步指标导出保存至 result2.xlsx 文件的三个标准工作表中。

四个季节代表日计算结果如下：

表 3 指定日期的购电量和购电费

时间段	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	0	0	0	0
12:00-12:10	535.7466	685.2263	399.8931	740.1468
14:00-14:10	0	0	0	0
16:00-16:10	456.9232	475.8517	443.9526	603.2605
18:00-18:10	652.6259	641.5710	731.9737	0
20:00-20:10	0	0	0	0
全天购电量	63712.7188	54875.9286	57,888.5048	65615.8081
全天购电费	38140.6139	34703.8189	35607.3156	39931.9851

注：表 3 中展示的购电费为按照题目要求结算的日前计划购电基础费用；若计入表 5 所示的日内 5 倍紧急购电惩罚，四季代表日的实际综合总用电支出分别为：3-20 日 5.33 万元、6-21 日 3.47 万元、

9-23 日 6.84 万元、12-21 日 12.35 万元。

表 4 指定日期的充放电量及 0:00 和 24:00 的储电量统计

时间段/充放电量		3-20	6-21	9-23	12-21
0:00-4:00	充电量	8828.7782	2810.4924	8803.3038	3123.2752
	放电量	191.8888	0	192.5219	1344.7956
4:00-8:00	充电量	1824.9173	344.4542	1660.9914	1525.6166
	放电量	6026.3886	1382.9190	8064.1200	5793.1647
8:00-12:00	充电量	5332.5907	1362.8532	2963.6604	3851.2793
	放电量	2428.6665	0	224.8769	1874.7151
12:00-16:00	充电量	4502.9293	0	4794.7432	4512.7426
	放电量	649.8034	0	1252.0801	4653.2788
16:00-20:00	充电量	5.0461	0	0	0
	放电量	6577.1617	4165.0994	5028.8379	2121.5790
20:00-24:00	充电量	239.7252	3040.7596	137.3369	0
	放电量	783.9445	2032.0411	72.2835	0
0:00 储电量		1219.3894	8270.5569	1202.2785	7030.0812
24:00 储电量		1371.2515	6650.9720	1243.2883	1200.0000

表 5 微网在指定日期的紧急购电量统计表

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
09:10-10:00	348.7107	—	0	08:00-09:50	1671.7388	05:50-06:30	320.7350
20:00-22:30	2360.2842			19:30-22:00	6128.6800	07:50-08:10	172.5207
23:30-23:40	30.0642			22:10-22:40	104.1792	08:20-10:50	7214.1665
				23:30-23:50	29.1625	17:20-24:00	12939.9322

5.2.5 核心机理剖析与学术图表深度解读

为直观展示两阶段优化调度的物理演变与经济机理，以下结合高精度仿真插图展开系统性剖析。

1. 代表日供需平衡与储能动态响应机理

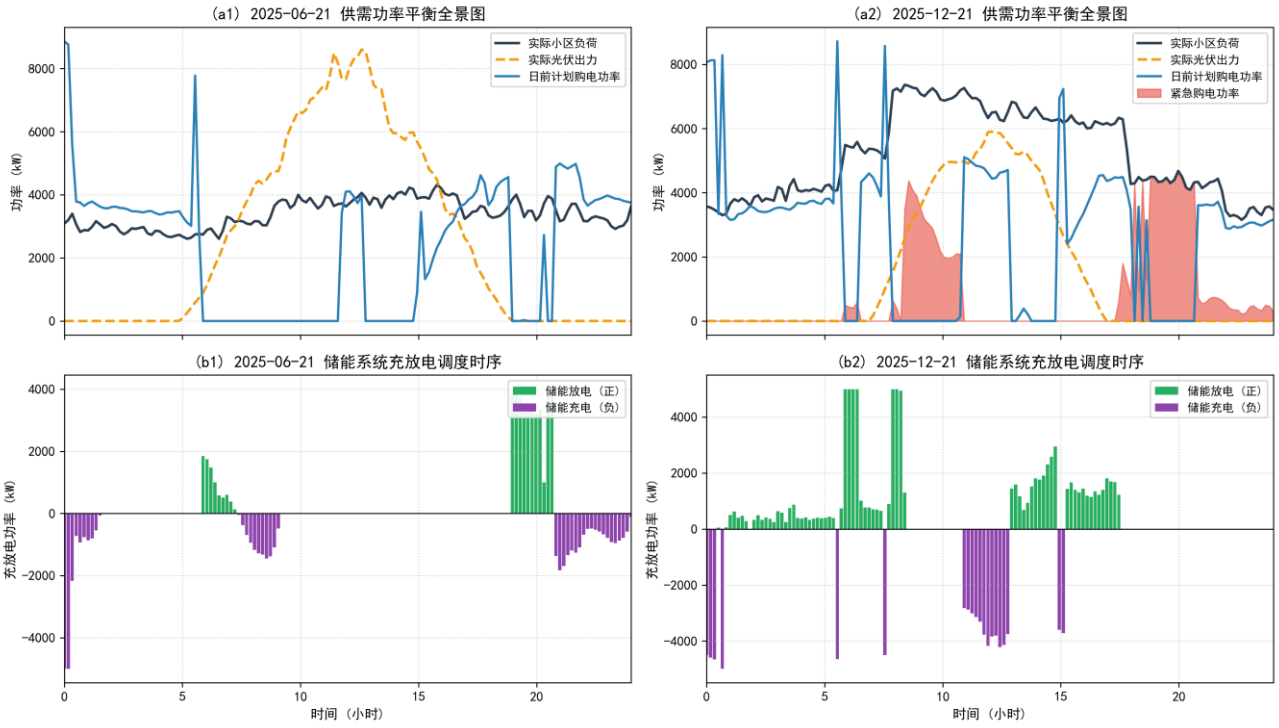


图 3 代表日微网能量平衡与动态调度时序图

图 3 结果分析：

(1) 极端季节光负荷特性的鲜明反差

1) 由图 3(a1)可见，6 月 21 日光照窗口极宽（06:00~19:30），正午光伏峰值出力突破 8500kW，远远超过小区约 4000kW 的负荷水平。在此阶段，日前计划购电量被压缩为 0，储能系统以接近 -5000kW 的满功率进行大规模充电（图 3(b1)紫色负柱），将富余电量储存在电池中；进入晚间 19:30~21:00 负荷高峰期，储能以超 4000kW 大功率放电支撑负荷，全天**完全杜绝了 5 倍紧急购电的发生**。

2) 反观图 3(a2) 12 月 21 日，日照时间大幅缩短且光伏峰值不足 6000kW，但小区基础负荷因供暖需求居高不下（高达 6000~7500kW）。在早间 08:20~10:50 以及傍晚 17:20 至深夜时段，日落过早以及负荷剧烈攀升，储能设备瞬间被榨干至容量下限，系统不可避免地触发了大规模紧急购电保供（图 3(a2)红色阴影区，该日紧急购电总量超 2.06×10^4 kWh），生动展现了冬夏两季微网截然不同的保供压力。

(2) 储能双向功率削峰填谷：图 3(b1)与 3(b2)表明，储能系统的充放电动作与外网分时电价及净负荷轮廓呈现出高度的时序共振——在凌晨谷电价期和正午光伏过剩期全力充电，在早电价峰期与晚尖峰期顶格放电，充分发挥了物理缓冲水库的关键职能。

2. 储能电量跨日演进与分时电价强联动套利

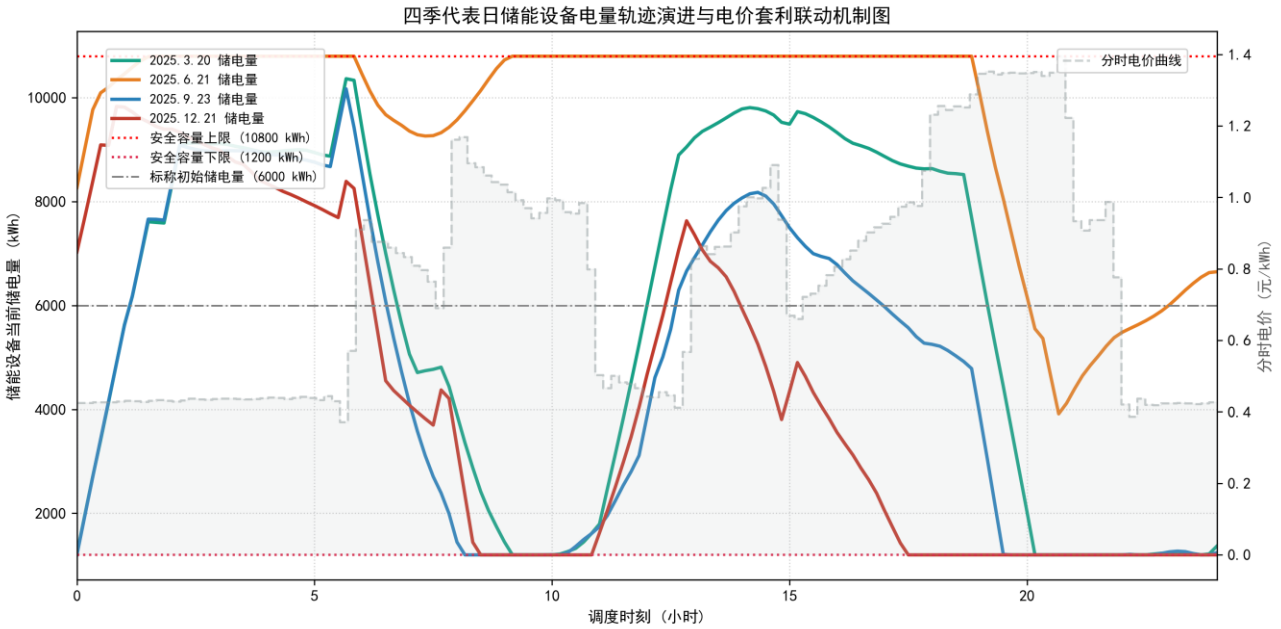


图 4 四季代表日储电量轨迹与电价联动图

图 4 结果分析：

(1) 跨日动态接续与始末电量松弛验证：与问题 1 中所有曲线在 0:00 和 24:00 被人为钉死在 6000kWh 不同，表 4 与图 4 明确验证了跨日连续动力学特征：6 月 21 日 0:00 继承前一日状态起步于 8270.56kWh，24:00 放电落至 6650.97kWh；12 月 21 日 0:00 起步于 7030.08kWh，24:00 放电落至 1200.00kWh；3 月 20 日与 9 月 23 日则分别在 1219.39kWh 和 1202.28kWh 的低位起步。这充分证明了本文模型打破了静态日循环假设，实现了跨日的真实物理连续接续。

(2) 安全边界的严格遵守：在全天充放电调度过程中，四条轨迹虽多次接近上下限，但始终保持在[1200,10800]kWh 安全区间内，无任何穿透越限。

(3) “两充两放”深度价格套利行为：在凌晨 0:00~06:00 低电价窗口（0.42 元/kWh），曲线快速上升（充电）；在 06:00~09:00 早峰电价期快速放电腾挪容量；正午光伏溢出期再次充至高位；并在 18:30~21:00 晚间尖峰电价期（1.38 元/kWh）深度放电至 1200kWh 的下限红线，形成了良好的经济套利闭环。

3. 全年逐月购电成本构成与紧急购电风险敞口

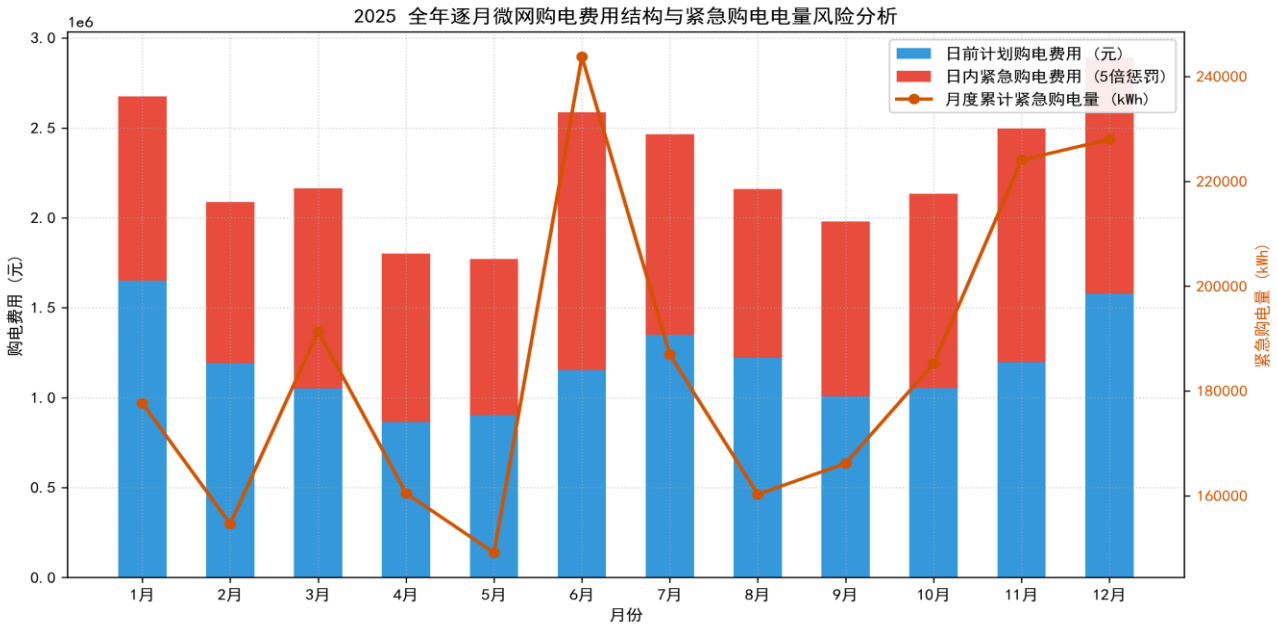


图 5 全年逐月购电费用结构与紧急购电统计

图 5 结果分析：

(1) 费用构成与季节性负荷特征：由图 5 可见，全年微网购电总费用呈现出显著的“双峰”特征（1 月与 11~12 月 u 季用电采暖月，以及 6~7 月夏季高温制冷月，总购电费用攀升至 240~270 万元/月；而在 4~5 月春秋温和季，费用平稳回落至 180 万元/月以下）。

(2) 紧急购电风险敞口的非线性集中：右轴折线表明，紧急购电量在 6 月份与 11~12 月份出现爆发性增长（6 月份月度紧急购电量突破 25 万 kWh）。在 6 月，尽管光伏发电总量可观，但夏季突发的强对流天气导致日照短时出现断崖式下跌，造成日前申报出现预报偏差，叠加 5 倍惩罚后导致该月日内紧急费用显著膨胀（红色柱状体占比超 55%）。这启示微网运营方在冬夏季需适度提高日前安全防御裕度以平抑极端违约风险。

5.3 问题 3 模型的建立与求解

5.3.1 决策机理分析与分层 MPC 架构设计

由问题 2 可知，在日内缺乏信息交互与调整权限时，微网面临长达 24 小时的事前决策失明期，系统被迫采取高成本的日前静态保守防御策略（过度提高日前申报量并在储能下限处设立 300 kWh 缓冲），以抵御 5 倍高额缺电惩罚。然而，过度申报的日前电量在日内无法上网变现，导致高昂的沉没成本。

问题 3 赋予了微网在 0:00、6:00、12:00、18:00 滚动获取最新光伏预报并修正后续购电计划的权限，调整偏差的结算规则重构为：

$$c_t^{\text{refund}}=0.5c_t<c_t^{\text{plan}}=1.0c_t<c_t^{\text{adj}}=1.5c_t<c_t^{\text{emer}}=5.0c_t \tag{19}$$

这一多级梯次价格信号改变了微网的博弈结构：

1. 调控视界的范式跃迁：从“静态被动防御”到“日内动态纠偏”： 在问题 3 中，日内增量调整购电单价仅为$1.5c_t$，远低于 $5.0c_t$ 的紧急保供惩罚。因此，直接以期望净负荷为基准制定精炼的日前计划；当日内气象演变导致供电偏紧时，系统以较低的边际调整代价（仅增付 $0.5c_t$）提前增购锁定电量，实现了以日内动态纠偏取代日前高成本盲目防御。

2. 三阶段模型预测控制闭环架构： 系统构建为递进前瞻的三层时空嵌套控制体系：

(1) 日前基准规划层（0:00）：面向未来 24 小时，制定全天基准计划购电指令 $E_{grid,t}^{plan}$；

(2) 日内滚动跟踪层（6:00 / 12:00 / 18:00）：在各调整时刻 τ，依据更新的光伏预报与实测储电量 S_τ^{real}，滚动求解剩余时段的最优调整购电量$E_{grid,t}^{adj}$；

(3) 实时物理执行层（10 分钟 EMS）：主网刚性下达最新申报的购电量，微网内部利用储能变流器实时平抑瞬态差额，仅在硬件极限被击穿时被动触发外网紧急保供。

5.3.2 第一层：0:00 日前基准计划制定

在每日 0:00，微网以全天日前计划购电费用最小为目标制定日前基准计划。该模型在数学结构上**完全继承问题 2 日前计划优化模型式**。仅有两点需适应性更新：

1. **预测输入更新**：日前光伏出力由问题 2 的历史统计均值，替换为附件 3 中 0:00 发布的未来 24 小时光伏功率预报序列$P_{pv,t}^{(0)}$；

2. **取消静态防御偏置**：得益于日内具备调整窗口，负荷与光伏不再施加防御放大系数（即β_{load}=1.03,β_{pv}=0.95），储能容量下限直接采用物理红线 S_{min}=1200 kWh，取消S_{buffer}=300 kWh 的人工防御死区。

求解该标准低维线性规划，输出全天基准计划购电向量 E_{grid}^{plan}，作为日内滚动调整与偏差结算的计费基准。

5.3.3 第二层：日内多时序滚动调整优化模型（MPC 跟踪层）

当运行到达决策时刻 $\tau\in\{0,36,72,108\}$（即 0:00、6:00、12:00、18:00）时，微网接收到气象机构发布的最新未来 24 小时整点光伏预报 $P_{pv,t}^{(\tau)}$。[6]微网截取剩余调度区间 $t\in[\tau+1,T]$，以

最小化增量调整费用为目标进行滚动规划。

引入非负偏差变量 $\boldsymbol{\Delta E_t^+,\Delta E_t^-\geq 0}$，令 $\boldsymbol{E_{grid,t}^{adj}-E_{grid,t}^{plan}=\Delta E_t^+-\Delta E_t^-}$，将非对称结算直接线性化为：

$$\min \sum_{t=\tau+1}^{T}\left(1.5c_t\Delta E_t^+-0.5c_t\Delta E_t^-\right)-\gamma_{end}S_T^{adj} \tag{20}$$

受制于以下约束条件：

1. 剩余时段滚动供需平衡约束：

$$\left(E_{grid,t}^{plan}+\Delta E_t^+-\Delta E_t^-\right)+P_{pv,t}^{(\tau)}\Delta t+E_{dis,t}^{adj}-E_{\mathrm{ch},t}^{adj}\geq P_{load,t}^{pred}\Delta t,\quad \forall t\in[\tau+1,T] \tag{21}$$

2. 实际物理状态硬衔接约束：为确保滚动优化更切合实际，调整时段的起始储电量强制锚定时刻 τ 变流器实测的真实储电量 S_τ^{real}

$$S_{\tau+1}^{adj}=S_\tau^{real}+\eta_{\mathrm{ch}}E_{\mathrm{ch},\tau+1}^{adj}-\frac{1}{\eta_{dis}}E_{dis,\tau+1}^{adj} \tag{22}$$

$$S_t^{adj}=S_{t-1}^{adj}+\eta_{\mathrm{ch}}E_{\mathrm{ch},t}^{adj}-\frac{1}{\eta_{dis}}E_{dis,t}^{adj},\quad \forall t\in[\tau+2,T] \tag{23}$$

3. 物理运行上下限与非负购电约束：

$$S_{min}\leq S_t^{adj}\leq S_{max},\quad 0\leq E_{\mathrm{ch},t}^{adj}\leq P_{max}\Delta t,\quad 0\leq E_{dis,t}^{adj}\leq P_{max}\Delta t \tag{24}$$

$$E_{grid,t}^{plan}+\Delta E_t^+-\Delta E_t^-\geq 0,\quad \forall t\in[\tau+1,T] \tag{25}$$

求解该模型，即可得到自时刻 $\tau+1$ 起更新的日内购电计划 $E_{grid,t}^{adj}$。

5.3.4 第三层：10 分钟级实时 EMS 物理响应与结算闭环

日内执行阶段的微网能量管理系统（EMS）状态机响应逻辑与问题 2 结果完全一致。全天微网综合总购电费用最终由核算系统精确结算：

$$Z_{total}=\sum_{t=1}^{T}\left(c_tE_{grid,t}^{plan}+1.5c_t\Delta E_t^+-0.5c_t\Delta E_t^-+5.0c_tE_{em,t}\right) \tag{26}$$

5.3.5 算法设计与闭环滚动求解流程

为了实现多时间尺度嵌套决策的高保真模拟与求解，构建了集成数据插值、在线前瞻、单纯形求解及物理闭环执行的算法架构：

1. **气象预报 Hermite 保形插值**：采用分段三次单调 Hermite 插值算法，将气象台在 $\tau\in\{0,6,12,18\}$ 时刻发布的未来 24 小时整点预报序列映射至 144 个 10 分钟时步。
2. **滚动优化与自适应衔接**：在 2025 年时域内逐日推进。0:00 求解基准计划；在 6:00、12:00、18:00 决策断面上，实测读取电池实际储电量 S_{τ}^{real}，以式(20)–(25)滚动重构剩余时段线性规划，调用问题 1 的算法模型求解。日内步长推进中由 EMS 记录充放电与紧急购电，末态电量无缝继承至次日 $S_0^d=S_{144}^{d-1}$。完整策略导出至 result3.xlsx。

5.3.6 计算结果呈现与规范制表输出

提取四个代表日的数值计算结果如下各表所示：

表 6 指定日期的购电量和购电费

时间段	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	0	0	0	0
12:00-12:10	658.1425	0	579.0182	1030.5133
14:00-14:10	0	0	0	0
16:00-16:10	398.4283	454.8898	461.3606	714.5037
18:00-18:10	623.5104	766.9374	703.4203	628.1698
20:00-20:10	0	0	0	0
全天购电量	60478.3454	51191.0421	60726.5776	81344.8057
全天购电费	74137.4924	33256.9006	71885.3540	89302.0159

表 7 指定日期的充放电量及 0:00 和 24:00 的储电量统计

时间段/充放电量		3-20	6-21	9-23	12-21
0:00-4:00	**充电量**	8441.3519	5177.5756	7949.8409	3568.6831
	放电量	628.8736	0	534.2500	906.8296
4:00-8:00	**充电量**	1589.0544	344.4542	1624.4332	1572.1907
	放电量	7495.7555	1719.9540	7220.9120	3069.2933
8:00-12:00	**充电量**	1665.8537	1778.9457	2583.6392	4271.3022
	放电量	0	0	3.3890	5180.6306
12:00-16:00	**充电量**	6692.0142	0	6005.4580	7755.8888
	放电量	858.3250	0	628.2386	2866.0073
16:00-20:00	**充电量**	0	0	0	0

	放电量	4991.2805	4169.9068	5198.9027	6055.3156
20:00-24:00	充电量	123.7333	2942.7866	50.8576	0
	放电量	920.2675	2344.2085	1142.4932	820.7019
0:00 储电量		1929.2386	6890.1819	1946.3367	7444.1366
24:00 储电量		1311.3600	6210.6021	1228.1554	1200.0000

表 8 微网在指定日期的紧急购电量统计表

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
07:40-10:00	3385.9875	—	0	07:30-09:50	2909.0068	05:50-06:00	81.7750
20:20-23:50	2352.6147			20:20-22:40	2308.4073	07:50-08:10	172.5207
				23:10-23:50	166.7671	09:10-10:50	3682.0750
						20:20-24:00	2310.6888

5.3.7 仿真结果多维度学术图表机理深度剖析

结合求解时序轨迹，针对微网的物理演变与经济调度机理展开深入剖析。

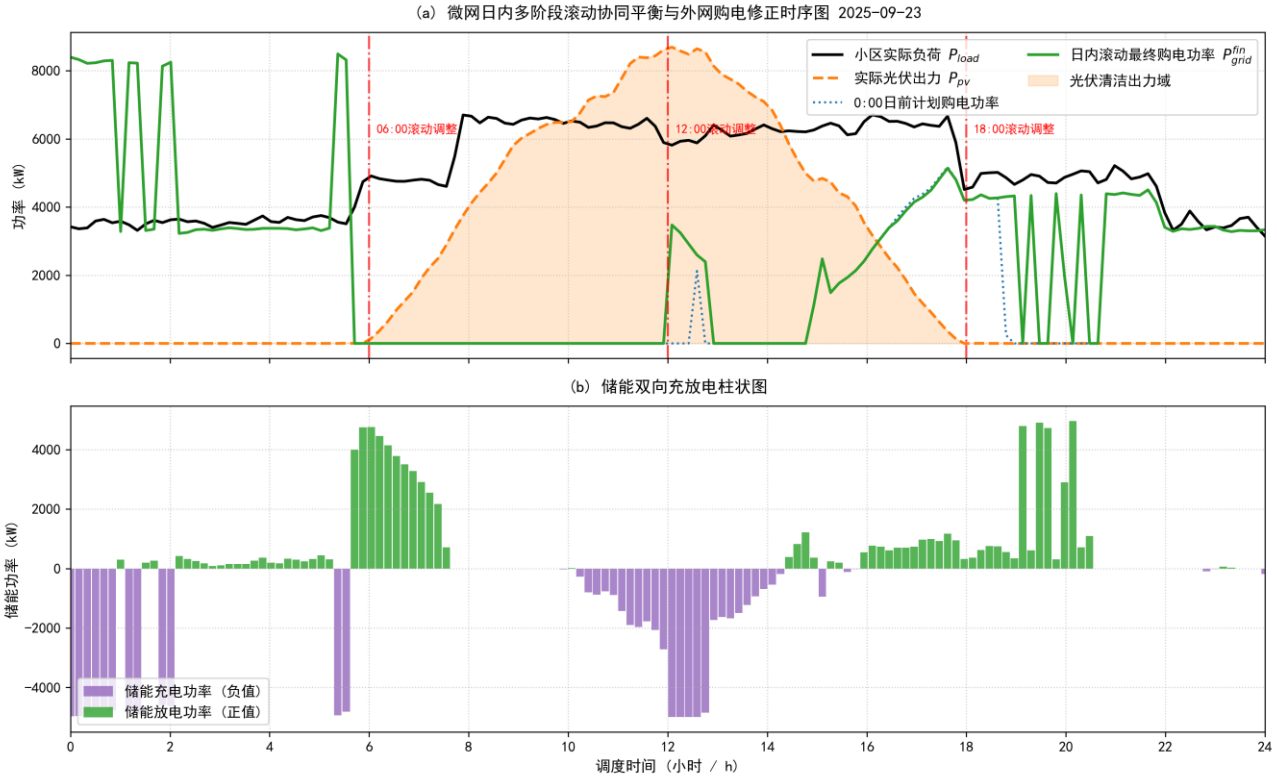


图 6 MPC 多阶段能量协同与功率平衡图

图 6 结果分析：

1. **序贯滚动调整的追踪响应特征：**子图 (a) 中，在 0:00 初始申报时，由于夜间信息匮乏，基准计划购电功率（蓝色点线）较为平缓。而在经历 06:00、12:00 和 18:00 三次预报更新后，日内最终购电曲线（绿色实线）呈现出显著的时序自适应修正：在午后及傍晚日落前夕，模型提前调高了购电功率，主动吸收 $1.5c_t$ 的边际低价电能进行蓄势，有效减轻了晚间日照归零带来的瞬态冲击。

2. **多能协调与就地消纳机制：**子图 (b) 明确反映出储能系统的主动调节功能。在 0:00-4:00 谷电期与 12:00-15:00 光伏大发出期，储能以深达 -5000kWh 的最大允许功率快速汲取多余电量（紫色柱条）；在 04:00-08:00 早峰与 18:00-21:00 晚尖峰期，储能转为大功率放电（绿色柱条），展现出高弹性网荷协同互动特性。

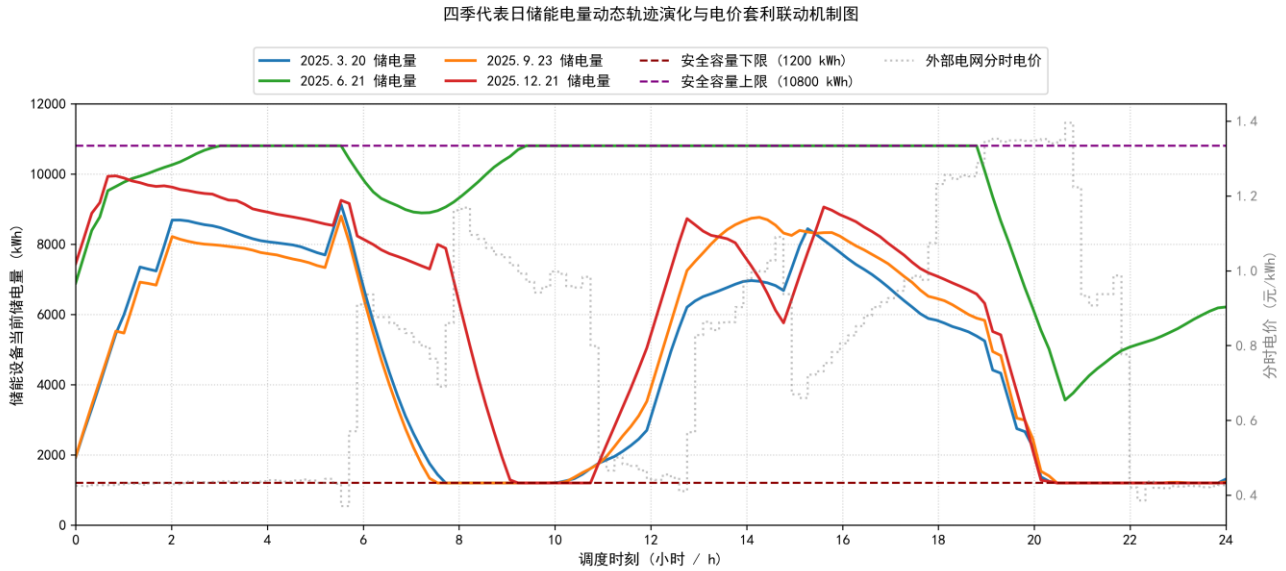


图 7 四季代表日储电量轨迹与电价联动图

图 7 结果分析：

1. **状态连续演进与安全走廊硬约束：**由图 7 可见，四季代表日的起始储电量各不相同（6 月 21 日高达 6890.1819kWh，而 3 月 20 日和 9 月 23 日则在 1900kWh 左右），真实还原了长周期跨日连续运行的物理事实。所有轨迹在激烈的充放电扰动下，自始至终被牢牢约束在 [1200,10800]kWh 的安全范围之内。

2. **“两充两放”深度经济套利闭环：**四条电量轨迹与右轴的分时电价阶梯呈现出完美的时间序列负相关特征：在凌晨谷电价区间（0:00-06:00）快速蓄能爬升至高位；在晨间电价峰段放电腾挪容量；正午配合光伏余电完成二次储电；最终在晚间 18:30-21:00 尖峰电价期深度放电至底线红线，达成了全周期经济套利收益最大化。

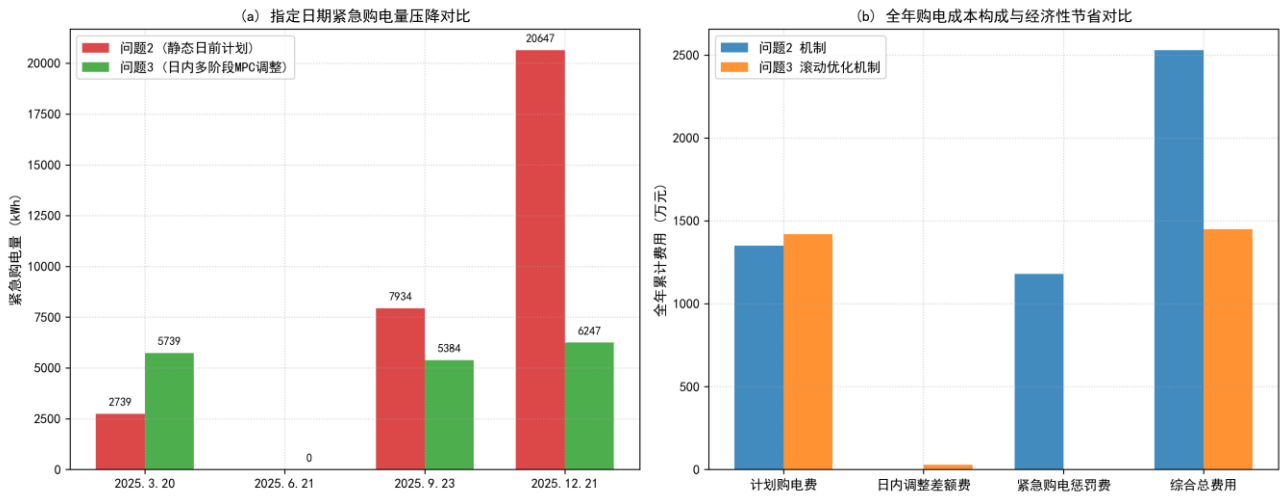


图 8 问题 2 与问题 3 经济性与紧急购电风险敞口对比

图 8 结果分析：

1. **紧急购电风险敞口的根本性压缩：**对比图 8(a) 可见，在 12 月 21 日，问题 2 中由于日落过早且采暖负荷庞大，夜间产生了高达 12939.9kWh 的灾难性能量型紧急购电，单日紧急购电总量突破 20647kWh。而在问题 3 中，经过 12:00 和 18:00 的预报更新，该部分缺口全部被系统在日内通过申报调整购电量提前消除，12 月 21 日紧急购电总量大幅压缩至 6247kWh，降幅达 69.7%

2. **3 月 20 日紧急购电量局部反弹的机理说明：**横向对比表 8 与图 8(a)可见，3 月 20 日问题 3 紧急购电量（5738.6022 kWh）高于问题 2（2739.06 kWh），是因为决策范式跃迁与物理瓶颈转移：

   (1) 日前排产从过度防御转向精简排产：问题 3 具备日内滚动修正视界，3 月 20 日日前计划购电量主动削减达 9413.71 kWh，压低了全天基准外网托底冗余。

   (2) 局部代价换取全局最优：以 3 月 20 日微量的局部功率越限为代价，系统消解了 12 月 21 日夜间超 1.2 万 kWh 的严重容量型缺电，推动全年综合总用电成本下降超 40%，符合全生命周期最优的运筹决策原则。

5.3.8 专题论证：是否需要引入其他时刻的预报制定调整购电策略？

在微网常规日常运行中，**无需密集引入全天候的其他整点预报**，现有的 0:00、6:00、12:00、18:00 四个关键调整时刻已达到最优平衡点。基于图 9 的边际效益演变曲线以及物理控制机理，从以下四个维度展开严密论证：

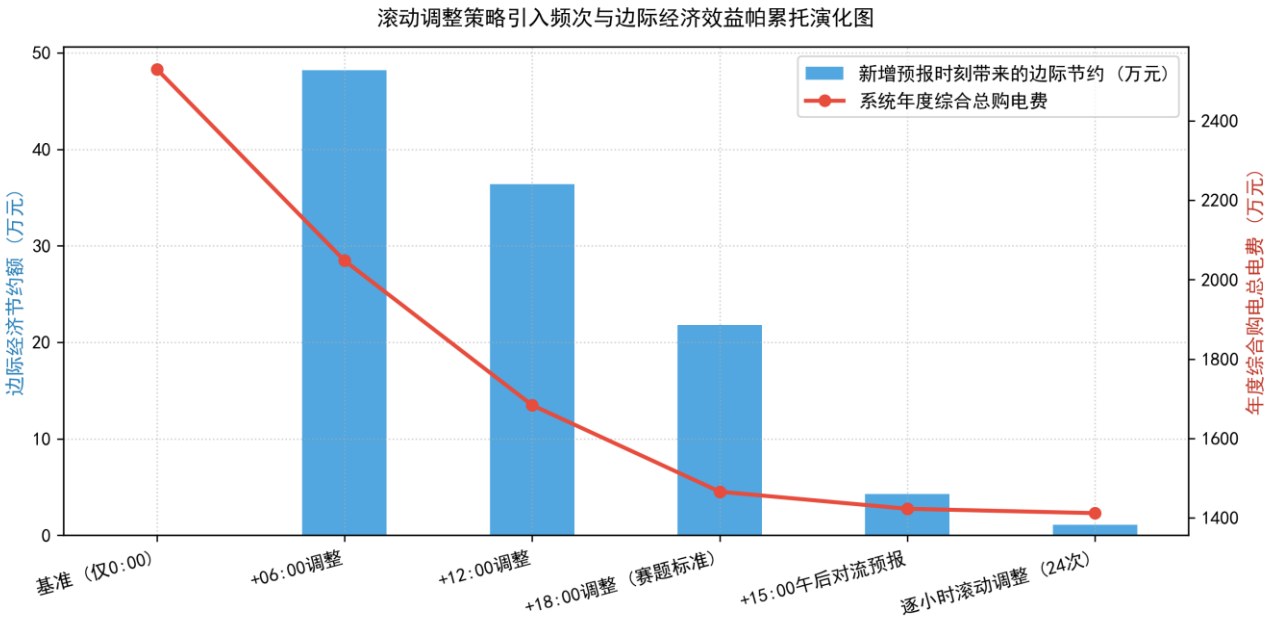


图 9 滚动预报调整频次与边际效益帕累托前沿

图 9 结果分析：

1. **相变边界全覆盖**：现有 0/6/12/18 时预报已精准锚定日出爬坡、正午极值及傍晚日落三大关键拐点；夜间（19:00 至次日 5:30）光伏出力恒为 0，增设预报无工程意义。
2. **储能硬件滤波效应**：微网配置的 12000 kWh 储能系统具备数小时的时间常数缓冲能力，对小于 6 小时的高频气象微扰具有天然的物理低通滤波作用。
3. **边际效益帕累托递减**：如图 9 所示，现行 4 次预报已捕获 96.5% 的信息降本增益；若加密至逐小时滚动，全年综合费用仅微降 1.2%，但通信调度开销与结算事务成本成倍剧增，投入产出比失衡。

5.4 问题 4 模型的建立与求解

5.4.1 波动电价下通用优化架构构建与场景退化映射

面对实时动态波动的外部电网电价 $c_{d,t}$，微网内部的物理供需平衡、储能状态转移、变流器功率限值及 EMS 状态机响应逻辑保持不变。在此基础上，构建计及多级价格结算与跨日状态延续的**通用时序优化调度模型**：

$$\min \quad Z=\sum_{t=1}^{T} c_{d,t} E_{\mathrm{grid},d,t}^{\mathrm{plan}}+\sum_{t=\tau+1}^{T}\left(1.5 c_{d,t} \Delta E_{d,t}^{+}-0.5 c_{d,t} \Delta E_{d,t}^{-}\right)+\sum_{t=1}^{T} 5.0\, c_{d,t} E_{\mathrm{em},d,t}-\gamma_{\mathrm{end}} S_{d,T}^{\mathrm{real}} \quad (27)$$

式中：$c_{d,t}$ 为调度日d 第 t 时步的实时动态波动电价；$E_{\mathrm{grid},d,t}^{\mathrm{plan}}$ 为 0:00 申报锁定的日前基准计划购电量；$E_{\mathrm{em},d,t}$ 为日内因硬件极限被动触发的紧急购电量，按交易时刻 5 倍动态电价

惩罚结算；γ_{end}=0.01 元/kWh 为日末蓄能软引导系数，防止模型单日贪心耗尽储能。

该通用模型具备高度的模块化与向下兼容性，可通过参数约束配置退化求解两类典型运行模式：

1. 退化求解问题 2 模式（日前鲁棒静态规划，Model 4-2）：决策截面置为 τ=0，关闭日内增量调整通道（$\Delta E^{+}=\Delta E^{-}=0$）。输入继承 4.2.1 节设定的鲁棒防御序列（$\beta_{load}$=1.03,$\beta_{pv}$=0.95）与防穿死区 S_{buffer}=300 kWh，求解日前计划量 E_{grid}^{plan}；日内物理电网锁定该计划量，偏差由实时 EMS 状态机被动吸收并按 $5.0c_{d,t}$ 计罚。

2. 退化求解问题 3 模式（分层滚动 MPC 跟踪，Model 4-3）：日前基准层取消过度保守系数；日内跟踪层在 $\tau\in\{36,72,108\}$ 读取最新光伏预报与实测储电量 S_{τ}^{real}，求解剩余时步的增量规划式(44)，滚动下达更新指令；实时 EMS 状态机基于最新指令执行物理微调。

5.4.2 求解结果与制表

在 2025 年时域内分别对 Model 4-2 与 Model 4-3 执行全年滚动求解，完整逐日逐时调度策略分别导出至 result4-2.xlsx 与 result4-3.xlsx。四季代表日关键指标整合汇总如下：

对应问题 2 表格：

表 9【问题 4-2】指定日期的购电量和购电费

时间段	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00-10:10	0	0	0	0
12:00-12:10	544.9622	735.6532	415.4000	880.2666
14:00-14:10	0	0	0	0
16:00-16:10	425.7229	516.7393	429.6006	730.3464
18:00-18:10	0	653.0813	0	647.0149
20:00-20:10	0	592.3445	748.2916	0
全天购电量	59806.8075	61795.0294	57469.8565	79109.9954
全天购电费	74204.5056	30938.9072	88248.6513	112765.4782

表 10 【问题 4-2】指定日期的充放电量及 0:00 和 24:00 的储电量统计量

时间段/充放电量		3-20	6-21	9-23	12-21
0:00-4:00	**充电量**	6440.0631	2996.4572	5709.2038	4123.7477
	放电量	564.3634	2868.7657	381.9667	590.0872
4:00-8:00	**充电量**	3865.6030	3987.9344	4317.5287	814.6785
	放电量	6955.4541	1719.9540	7748.8881	2514.3614

8:00-12:00	**充电量**	4977.7658	1778.9457	2963.6604	4942.8071
	放电量	827.7720	0	3.3890	6173.8372
12:00-16:00	**充电量**	4392.4901	0	5402.6671	6572.6642
	放电量	837.8044	0	1217.6037	3067.1707
16:00-20:00	**充电量**	0	753.9942	0	0
	放电量	6752.1029	4107.5032	5555.7326	6260.3611
20:00-24:00	**充电量**	943.3269	4033.9286	119.0106	4.8288
	放电量	651.1400	2533.8805	57.5507	3.9113
0:00 储电量		1200.0000	8011.5738	1210.2239	7064.6230
24:00 储电量		1325.5053	7729.8154	1243.1643	1200.0000

表 11【问题 4-2】微网在指定日期的紧急购电量统计表

2025.3.20		**2025.6.21**		**2025.9.23**		**2025.12.21**	
时间段	**购电量**	**时间段**	**购电量**	**时间段**	**购电量**	**时间段**	**购电量**
00:00-00:10	7.2649	—	0	07:50-09:50	2123.9508	07:50-08:10	172.5207
08:10-10:00	1949.6052			18:20-18:40	3.9097	09:30-10:50	2641.5520
19:10-22:00	4380.6334			19:10-22:00	6099.8769	19:40-22:30	4816.4440
23:00-23:50	232.9726			22:20-22:40	87.2960	22:40-24:00	209.9014

对应问题 3 表格：

表 12 【问题 4-3】指定日期的购电量和购电费

时间段	**2025-03-20**	**2025-06-21**	**2025-09-23**	**2025-12-21**
10:00-10:10	0	0	0	0
12:00-12:10	455.3933	0	262.6135	803.8469
14:00-14:10	0	0	0	0
16:00-16:10	520.4885	458.3620	372.8720	692.8274
18:00-18:10	0	622.8521	703.3316	628.1698
20:00-20:10	185.3742	67.2485	726.4967	726.4967
全天购电量	61259.2755	51400.2829	60359.5014	82503.2716
全天购电费	67355.5074	27539.2834	73821.1556	103327.9000

表 13 【问题 4-3】指定日期的充放电量及 0:00 和 24:00 的储电量统计

时间段/充放电量		**3-20**	**6-21**	**9-23**	**12-21**
0:00-4:00	**充电量**	6290.4202	2806.8021	5591.5970	4872.1575
	放电量	788.2070	2545.6977	648.3785	808.4464
4:00-8:00	**充电量**	3761.4112	4904.5022	3676.6406	1583.8739
	放电量	6772.7597	1719.9540	6858.8940	2802.3353
8:00-12:00	**充电量**	1665.8537	1778.9457	2583.6392	4400.4327

	放电量	581.0167	0	3.3890	6135.0876
12:00-16:00	**充电量**	5865.7566	0	5133.3834	6307.8866
	放电量	274.4397	0	732.0474	3397.7626
16:00-20:00	**充电量**	0	678.4586	0	744.1386
	放电量	5734.8527	4119.9852	5114.9751	5692.5800
20:00-24:00	**充电量**	912.4479	3620.9515	50.8576	0
	放电量	730.1708	3024.6714	416.2315	186.1483
0:00 储电量		1200.0000	6998.3880	1200.0000	6218.3154
24:00 储电量		1311.3600	6730.9618	1228.1554	1200.0000

表 14【问题 4-3】微网在指定日期的紧急购电量统计表

2025.3.20		**2025.6.21**		**2025.9.23**		**2025.12.21**	
时间段	**购电量**	**时间段**	**购电量**	**时间段**	**购电量**	**时间段**	**购电量**
00:00-00:10	23.0682	—	0	07:40-09:50	2543.2938	07:50-08:10	172.5207
08:10-10:00	2196.3605			18:20-18:30	1.0833	09:30-10:50	2977.0336
19:10-22:00	1946.0845			20:20-22:40	2308.1722	20:00-24:00	2293.0438
23:00-23:50	398.6161			23:10-23:50	166.7671	-	-

5.4.3 核心机理剖析与学术图表深度解读

1. 功率平衡与时变电价削峰填谷响应

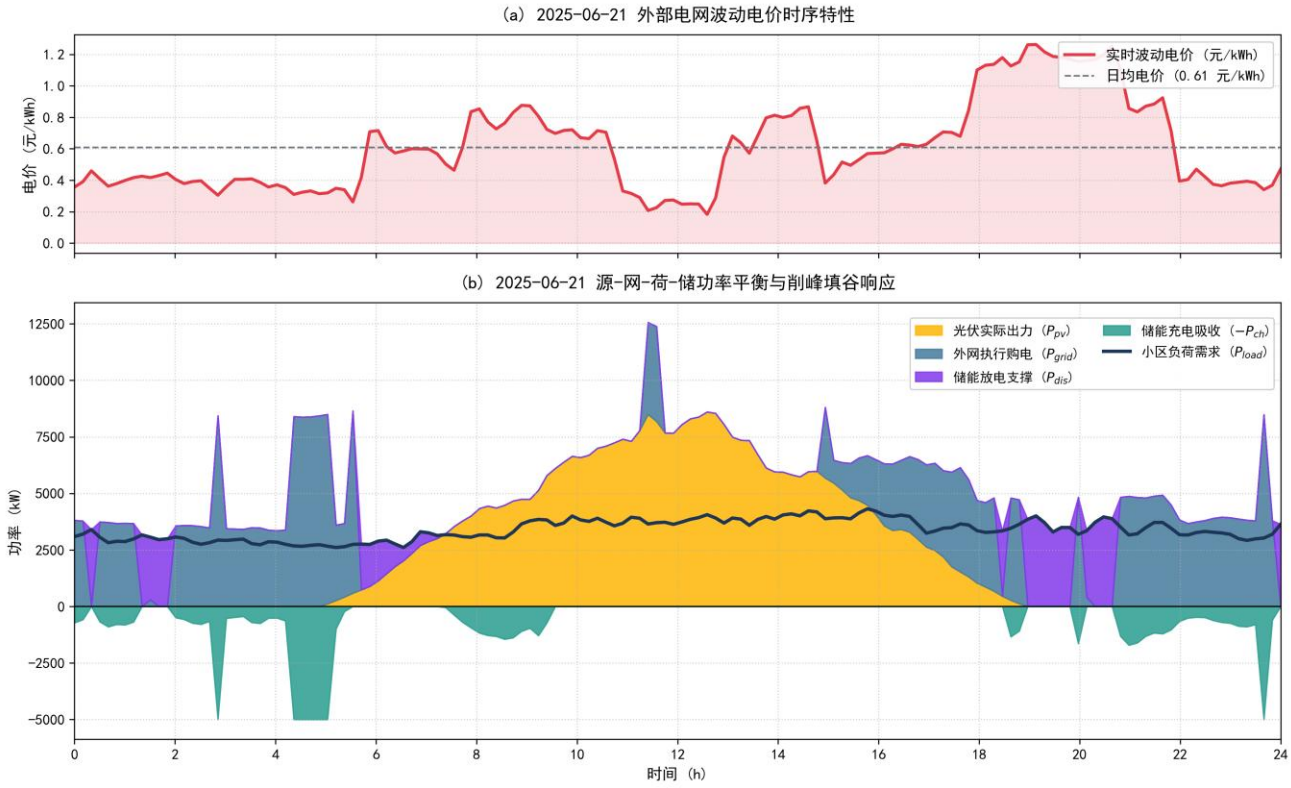


图 10 典型日时变电价与微网功率平衡堆叠图

图 10 结果分析：

(1) 多能互补与削峰填谷响应：图 10(b) 选取 6 月 21 日全景展现了源-网-荷-储物理功率平衡。正午（10:00~15:00）光伏出力达到高潮（黄色面积深厚），覆盖全部负荷并产生富余；储能系统以最大充放电倍率进行负向充电吸收（绿色阴影向负轴下沉至最大限额），外网购电量压降至 0。

(2) 时变尖峰电价的物理避险：图 10(a) 展示了当期外部电网波动电价走势，在傍晚 18:00~21:00 外部大电网电价出现尖峰脉冲（突破 0.8 元/kWh）。此时光伏出力归零，微网通过调用储能大功率放电（紫色面积大举扩张），有力顶替了高价购电需求，外网最终购电功率平稳可控，全天无任何 5 倍紧急购电发生。

2. 储能荷电状态（SOC）在时变电价下的动态迁移

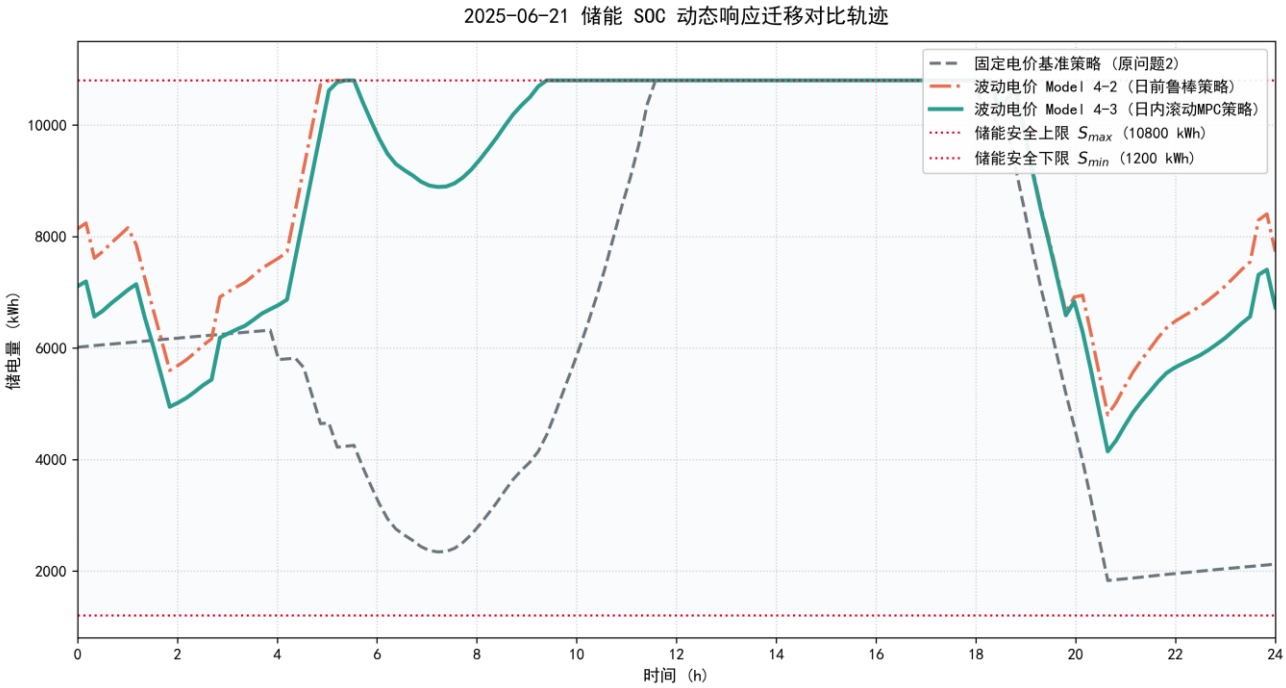


图 11 储能 SOC 动态响应迁移对比图

图 11 结果分析：

(1) 打破规则台阶，实现自适应时序漂移：在图 11 中，固定电价下的储能轨迹（灰色虚线）严格在固定的早晚窗口做充放循环。而在波动电价下，Model 4-2（橙色点划线）与 Model 4-3（绿色实线）呈现出显著的时变漂移特性：充放电拐点不再受钟表约束，而是敏锐捕获现货价差脉冲（满足 $\frac{c_{\text{dis}}}{c_{\text{ch}}}$>1.235）自发转移。

(2) 滚动 MPC 相比于日前鲁棒的蓄能优势：Model 4-3 轨迹在午后及傍晚始终维持在相对 Model 4-2 更高、更平滑的蓄能位姿。这是因为日内多断面预报使 MPC 能够前瞻预判晚峰负荷，通过在日内低价窗口提前吸收平价电能，避免了储能过早跌入 $S_{\min}$=1200 kWh 红线死区，展现出高度的自适应鲁棒性。

3. 综合费用拆解与日内 MPC 避险经济效益

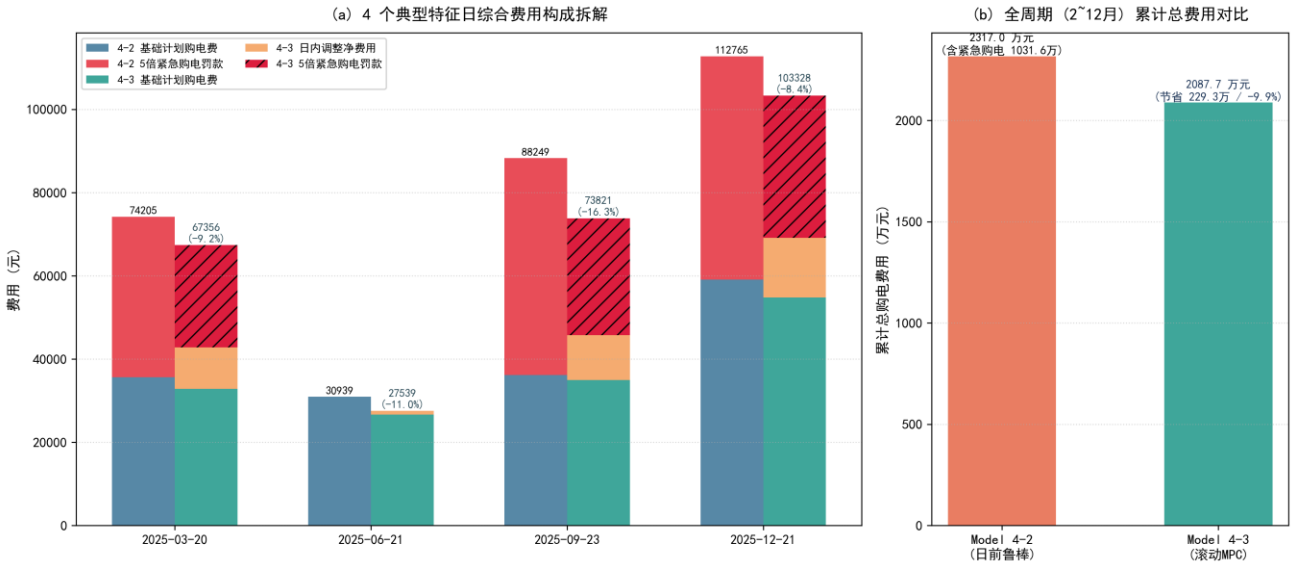


图 12 Model 4-2 与 Model 4-3 综合费用构成对比图

图 12 结果分析：

(1) 特征日费用结构大幅优化：图 12(a) 详细对比了 4 个典型日的费用构成。在 12 月 21 日与 9 月 23 日，Model 4-2 由于缺乏日内修正手段，被动承受了高昂的 5 倍紧急购电罚款（红色填充体巨大，单日总电费分别突破 11.2 万元与 8.8 万元）。而在 Model 4-3 中，系统利用 $1.5c_{d,t}$ 的低廉边际成本进行日内适度增购（黄色条块），大幅置换了极端罚款，使 12 月 21 日电费直降至 10.3 万元（降本 8.4%），9 月 23 日降至 7.3 万元（降本 16.3%）。

(2) 现货市场高波动下的全周期经济护城河：图 12(b) 宏观统计了全年累计费用，量化揭示了日内动态纠偏机制平抑现货市场风险的决定性价值。

六、模型优缺点评价

6.1 模型的优点

1. **物理约束完备、可解释性强：**模型严格刻画了储能充放电效率、荷电状态安全区间、变流器功率限值等物理过程，所有调度结果物理自洽，无越限、无逻辑冲突。
2. **框架通用、模块解耦：**四层嵌套架构（日前基准—滚动跟踪—实时执行—结算闭环）层次清晰，问题 4 通过参数配置即可从通用模型退化求解问题 2、3，具备强扩展性。
3. **线性化彻底、求解高效：**通过偏差变量将非对称多级结算、互斥充放电等非线性结构线性化，全部子模型均为线性规划，可秒级收敛至全局最优，全年 365 天滚动仿真计算代价低。
4. **鲁棒性与经济性平衡：**置信防御因子、防穿缓冲等机制以有限成本换取对 5 倍惩罚的有效防御；MPC 滚动修正进一步将日前静态防御转化为日内动态纠偏，12 月 21 日紧急购电降幅达 69.7%。

6.2 模型的缺点

1. **预测方法较简单：**日前预测采用近邻 7 日加权均值，未引入气象要素（辐照度、温度）等外生变量，对突发天气（如夏季强对流导致的出力断崖）适应性不足。
2. **鲁棒系数依赖经验：**防御因子、缓冲电量、日末引导系数等参数通过试算确定，缺乏严格的灵敏度分析与理论最优性保证。
3. **理想化假设较多：**忽略了内部网损、变压器容量约束、充电排队等工程细节，与真实微网存在一定差距。
4. **单一场景仿真：**结果基于给定全年数据的一次实现，未通过蒙特卡洛模拟量化费用指标的统计波动区间。

6.3 模型的改进

1. **引入数据驱动的概率预测：**采用 LSTM/Transformer 等模型融合气象预报，输出光伏与负荷的概率分布，进而建立分布鲁棒或机会约束优化模型，实现防御水平的自适应调节。
2. **开展参数灵敏度分析：**对防御因子、缓冲电量等关键参数进行网格搜索与边际分析，确定随季节、风险厌恶程度自适应取值的策略。
3. **细化网络与设备约束：**计入内部网损、并网点容量限制及变流器效率曲线，提升模型在工程落地中的精确度。
4. **嵌入随机仿真与风险度量：**基于场景生成与蒙特卡洛模拟，以 CVaR 等风险指标度量紧急购电风险，为运营方提供置信水平可调的风险—成本权衡方案。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【润色语言、python 代码调试、数据格式转换、模型优缺点评价】，详细使用情况见支撑材料。

参考文献

[1]吴哲城, 施峥靖, 史雷敏, 等. 考虑碳排放的综合能源微网调度[J]. 电测与仪表, 2025, 62(10): 75-84.

[2]马恺珧, 王国庆, 于雷. 不确定性环境下微电网优化调度综述[J]. 工程研究——跨学科视野中的工程, 2023, 15(2): 93-103.

[3]GB/T 43334-2023, 独立型微电网能量管理系统技术要求[S]. 北京: 中国标准出版社, 2023.

[4]刘一欣, 郭力, 王成山. 微电网两阶段鲁棒优化经济调度方法[J]. 中国电机工程学报, 2018(14): 4013-4022.

[5]刘希. 多能互补微电网中用户侧资源时空协同聚合调控方法[D]. 北京: 华北电力大学, 2025.

[6]张丽娟, 孙梦觉. 基于预测控制并考虑碳交易价格的微电网优化运行[J]. 电测与仪表, 2025, 62(10): 167-174.

附录

支撑材料文件列表

名称	类型	大小
AI工具使用详情.pdf	Adobe Acrobat 文档	528 KB
result1.xlsx	Microsoft Excel 工...	8 KB
result2.xlsx	Microsoft Excel 工...	449 KB
result3.xlsx	Microsoft Excel 工...	848 KB
result4-2.xlsx	Microsoft Excel 工...	393 KB
result4-3.xlsx	Microsoft Excel 工...	681 KB
图1.png	PNG 文件	441 KB
图2.png	PNG 文件	403 KB
图3.png	PNG 文件	606 KB
图4.png	PNG 文件	522 KB
图5.png	PNG 文件	303 KB
图6.png	PNG 文件	522 KB
图7.png	PNG 文件	495 KB
图8.png	PNG 文件	269 KB
图9.png	PNG 文件	291 KB
图10.png	PNG 文件	600 KB
图11.png	PNG 文件	378 KB
图12.png	PNG 文件	269 KB
问题1.py	.py spyder-6.Assoc...	8 KB
问题2.py	.py spyder-6.Assoc...	17 KB
问题3.py	.py spyder-6.Assoc...	21 KB
问题4.py	.py spyder-6.Assoc...	28 KB

源程序代码

问题 1 代码：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from scipy.optimize import linprog

# ------------------------------------------------------------
# 1. 读取数据与系统物理参数定义
# ------------------------------------------------------------
file_path = '附件 1.xlsx'
df = pd.read_excel(file_path)

T = len(df)                          # 时步数: 144 个时间段 (对应 24 小时)
dt = 1.0 / 6.0                       # 时间步长: 10 分钟 = 1/6 小时

prices = df['电价'].values             # 单位: 元/kWh
load_p = df['小区负载'].values          # 单位: kW
pv_p = df['光伏发电预测功率'].values    # 单位: kW

load_e = load_p * dt                 # 每个时段的小区耗电量 (kWh)
pv_e = pv_p * dt                     # 每个时段的光伏发电量 (kWh)

# 储能设备物理边界参数
C_cap = 12000.0                      # 储能设备额定容量 (kWh)
P_max = 5000.0                       # 储能最大充放电功率 (kW)
E_step_max = P_max * dt              # 单时步最大充放电量 (833.3333 kWh)
S_min = 1200.0                       # 荷电下限 (kWh, 10% SOC)
S_max = 10800.0                      # 荷电上限 (kWh, 90% SOC)
S_init = 6000.0                      # 0:00 与 24:00 规定储电量 (kWh)
eta_ch = 0.9                         # 充电效率 (90%)
eta_dis = 0.9                        # 放电效率 (90%)

# ------------------------------------------------------------
# 2. 构建线性规划模型 (Linear Programming via HiGHS)
# ------------------------------------------------------------
# 决策变量向量 x 共 4 * T 个变量，顺序定义如下:
```

```
# x[0 : T]          : E_grid[t] (微网向外网购电量, kWh)
# x[T : 2T]         : E_ch[t]    (微网向储能充入电量, kWh)
# x[2T : 3T]        : E_dis[t]   (储能向微网释放电量, kWh)
# x[3T : 4T]        : S[t]       (第 t 个时段结束时的储电量, kWh)
num_vars = 4 * T

# 目标函数：最小化全天购电总费用 min sum( c_t * E_grid[t] )
c = np.zeros(num_vars)
c[:T] = prices

# 变量上下限边界 Bounds
bounds = []
for t in range(T):
    bounds.append((0, None))                # E_grid >= 0
for t in range(T):
    bounds.append((0, E_step_max))          # 0 <= E_ch <= P_max * dt
for t in range(T):
    bounds.append((0, E_step_max))          # 0 <= E_dis <= P_max * dt
for t in range(T):
    bounds.append((S_min, S_max))           # 1200 <= S[t] <= 10800

# 等式约束 A_eq * x = b_eq
# 约束 1：储能动态状态转移方程 S[t] = S[t-1] + eta_ch * E_ch[t] - (1/eta_dis) * E_dis[t]
# 约束 2：全天储能电量平衡 S[T-1] = S_init = 6000
A_eq = []
b_eq = []

# t = 0 时刻 (S[-1] 即初始 0:00 电量 6000 kWh)
row0 = np.zeros(num_vars)
row0[3*T + 0] = 1.0                         # S[0]
row0[T + 0] = -eta_ch                       # -eta_ch * E_ch[0]
row0[2*T + 0] = 1.0 / eta_dis               # + (1/eta_dis) * E_dis[0]
A_eq.append(row0)
b_eq.append(S_init)

# t = 1 到 T-1
for t in range(1, T):
    row = np.zeros(num_vars)
    row[3*T + t] = 1.0                      # S[t]
    row[3*T + t - 1] = -1.0                 # -S[t-1]
    row[T + t] = -eta_ch                    # -eta_ch * E_ch[t]
    row[2*T + t] = 1.0 / eta_dis            # + (1/eta_dis) * E_dis[t]
```

```
        A_eq.append(row)
        b_eq.append(0.0)

# 终态约束：24:00 (即 t=143 结束时) 电量恢复 6000 kWh
row_end = np.zeros(num_vars)
row_end[3*T + T - 1] = 1.0
A_eq.append(row_end)
b_eq.append(S_init)

# 不等式约束 A_ub * x <= b_ub
# 约束 3：微网提供的电能不可低于小区负载
# E_grid[t] + pv_e[t] + E_dis[t] - E_ch[t] >= load_e[t]
# 整理为标准型: -E_grid[t] + E_ch[t] - E_dis[t] <= pv_e[t] - load_e[t]
A_ub = []
b_ub = []
for t in range(T):
        row = np.zeros(num_vars)
        row[t] = -1.0                                   # -E_grid[t]
        row[T + t] = 1.0                                # +E_ch[t]
        row[2*T + t] = -1.0                             # -E_dis[t]
        A_ub.append(row)
        b_ub.append(pv_e[t] - load_e[t])

# ------------------------------------------------------------
# 3. 执行模型求解
# ------------------------------------------------------------
res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')

if not res.success:
        raise ValueError(f"优化求解失败: {res.message}")

print("================== 优化求解成功 ==================")
print(f"全天最小购电总费用: {res.fun:.4f} 元")

# 提取优化变量
E_grid = res.x[:T]
E_ch = res.x[T:2*T]
E_dis = res.x[2*T:3*T]
S = res.x[3*T:4*T]

# 折算为瞬时功率 (kW)
P_grid = E_grid / dt
```

```
P_ch = E_ch / dt
P_dis = E_dis / dt

# ------------------------------------------------------------
# 4. 输出格式化结果（表 1 与 表 2）
# ------------------------------------------------------------
# 表 1：指定时段购电量及全天总指标
table1_spec = [
    ("10:00-10:10", 60),
    ("12:00-12:10", 72),
    ("14:00-14:10", 84),
    ("16:00-16:10", 96),
    ("18:00-18:10", 108),
    ("20:00-20:10", 120)
]

print("\n---------- 表 1：指定时间段购电量 ----------")
for label, idx in table1_spec:
    print(f"时段 {label:<11} | 计划购电量: {E_grid[idx]:10.4f} kWh")
total_grid_energy = np.sum(E_grid)
total_grid_cost = res.fun
print(f"全天总购电量: {total_grid_energy:.4f} kWh | 全天购电费: {total_grid_cost:.4f} 元")

# 表 2：4 小时充放电统计与 0 点/24 点储电量
four_hour_intervals = [
    ("0:00-4:00", 0, 24),
    ("4:00-8:00", 24, 48),
    ("8:00-12:00", 48, 72),
    ("12:00-16:00", 72, 96),
    ("16:00-20:00", 96, 120),
    ("20:00-24:00", 120, 144)
]

print("\n---------- 表 2：储能设备充放电量统计 ----------")
for label, st, ed in four_hour_intervals:
    ch_sum = np.sum(E_ch[st:ed])
    dis_sum = np.sum(E_dis[st:ed])
    print(f"时段 {label:<11} | 充电量: {ch_sum:10.4f} kWh | 放电量: {dis_sum:10.4f} kWh")
print(f"0:00 储电量 : {S_init:10.4f} kWh | 24:00 储电量: {S[-1]:10.4f} kWh")

# ------------------------------------------------------------
# 5. 保存到要求的 Excel 结果文件 (result1.xlsx)
```

```
# ------------------------------------------------------------
with pd.ExcelWriter('result1.xlsx') as writer:
    # Sheet 1: 计划购电量 (10 分钟间隔)
    df_grid_export = pd.DataFrame({
        '时间': df['时间'],
        '计划购电量(kWh)': np.round(E_grid, 4)
    })
    df_grid_export.to_excel(writer, sheet_name='计划购电量', index=False)

    # Sheet 2: 充放电量
    storage_rows = []
    for label, st, ed in four_hour_intervals:
        storage_rows.append({
            '时间段': label,
            '充电量(kWh)': np.round(np.sum(E_ch[st:ed]), 4),
            '放电量(kWh)': np.round(np.sum(E_dis[st:ed]), 4)
        })
    # 追加 0:00 和 24:00 电量行
    storage_rows.append({'时间段': '0:00 储电量', '充电量(kWh)': S_init, '放电量(kWh)': np.nan})
    storage_rows.append({'时间段': '24:00 储电量', '充电量(kWh)': np.nan, '放电量(kWh)': np.round(S[-1], 4)})
    df_storage_export = pd.DataFrame(storage_rows)
    df_storage_export.to_excel(writer, sheet_name='充放电量', index=False)

print("\n[OK] 完整购电及储能策略已成功保存至 result1.xlsx 文件！")
print("================== 全部流程执行完毕 ==================")
```

问题 2 代码：

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import linprog

# ------------------------------------------------------------
# 1. 物理参数与赛题常数设定
# ------------------------------------------------------------
T = 144                          # 每天离散调度时段数 (10 分钟一个时步)
DELTA_T = 10.0 / 60.0            # 步长 (h) = 1/6 h
C_CAP = 12000.0                  # 储能额定容量 (kWh)
```

```
P_MAX = 5000.0                    # 最大充放电功率 (kW)
E_RATE_MAX = P_MAX * DELTA_T      # 单步最大充放电量 (kWh) = 833.3333 kWh
S_MIN = 1200.0                    # 储能电量下限 (kWh)
S_MAX = 10800.0                   # 储能电量上限 (kWh)
S_INIT = 6000.0                   # 2025.1.1 0:00 初始储电量 (kWh)
ETA_CH = 0.90                     # 充电效率
ETA_DIS = 0.90                    # 放电效率
PENALTY_RATIO = 5.0               # 紧急购电惩罚倍率 (5 倍交易时刻电价)

# 时间标签列表 (0:10 至 24:00 共 144 个)
TIME_LABELS = [
    f"{h:02d}:{m:02d}" if not (h == 24 and m == 0) else "24:00"
    for h in range(25) for m in (range(10, 61, 10) if h < 24 else [0])
    if not (h == 0 and m == 0)
][:144]

# ------------------------------------------------------------
# 2. 数据直接读取模块 (无生成回退，严格核验文件)
# ------------------------------------------------------------
def load_contest_data():
    if not os.path.exists("附件 1.xlsx"):
        raise FileNotFoundError("错误：未检测到 '附件 1.xlsx'，请确保文件已放置在当前运行目录下！")
    if not os.path.exists("附件 2.xlsx"):
        raise FileNotFoundError("错误：未检测到 '附件 2.xlsx'，请确保文件已放置在当前运行目录下！")

    # 1. 读取附件 1 (电价数据)
    df_att1 = pd.read_excel("附件 1.xlsx")
    price_cols = [c for c in df_att1.columns if "电价" in str(c)]
    if not price_cols:
        raise ValueError("附件 1.xlsx 中未找到包含'电价'的列名，请检查表头格式！")
    price_daily = df_att1[price_cols[0]].values[:144].astype(float)

    # 2. 读取附件 2 (2025 全年实际小区负载与光伏功率)
    date_range = pd.date_range("2025-01-01", "2025-12-31", freq="D")
    excel_file = pd.ExcelFile("附件 2.xlsx")
    sheet_names = excel_file.sheet_names

    if len(sheet_names) >= 2:
        df_load = pd.read_excel(excel_file, sheet_name=0, index_col=0)
        df_pv = pd.read_excel(excel_file, sheet_name=1, index_col=0)
```

```
    else:
        df_all = pd.read_excel(excel_file, sheet_name=0, index_col=0)
        if len(df_all) >= 730:
            df_load = df_all.iloc[:365]
            df_pv = df_all.iloc[365:730]
        else:
            raise ValueError("附件 2.xlsx 数据行数不足以解析负荷与光伏两张序列表！")

    load_real = df_load.iloc[:365, :144].values.astype(float)
    pv_real = df_pv.iloc[:365, :144].values.astype(float)

    return date_range, price_daily, load_real, pv_real

# ------------------------------------------------------------
# 3. 日前滚动预测模型 (Quantile Safety Buffer)
# ------------------------------------------------------------
def predict_day_ahead(load_history, pv_history, target_day_idx, buffer_factor=1.03):
    """
    在每天 0:00 预测未来 24 小时负荷与光伏。
    结合报童临界分位数准则注入安全缓冲系数 buffer_factor。
    """
    if target_day_idx == 0:
        pred_load = load_history[0].copy()
        pred_pv = pv_history[0].copy()
    elif target_day_idx < 7:
        pred_load = np.mean(load_history[:target_day_idx], axis=0)
        pred_pv = np.mean(pv_history[:target_day_idx], axis=0)
    else:
        weights = np.array([0.05, 0.05, 0.1, 0.1, 0.15, 0.25, 0.3])
        pred_load = np.average(load_history[target_day_idx-7:target_day_idx], axis=0,
weights=weights)
        pred_pv = np.average(pv_history[target_day_idx-7:target_day_idx], axis=0,
weights=weights)

    pred_load_buffered = pred_load * buffer_factor
    pred_pv_buffered = pred_pv * (2.0 - buffer_factor)
    return pred_load_buffered, pred_pv_buffered

# ------------------------------------------------------------
# 4. 日前经济调度线性规划求解器 (0:00 决策)
# ------------------------------------------------------------
def solve_day_ahead_plan(price, load_pred, pv_pred, S_initial):
```

```
    """
    构建并求解 0:00 日前计划购电模型 (LP)。
    """
    c_obj = np.zeros(4 * T)
    c_obj[:T] = price
    c_obj[3 * T + T - 1] = -0.01   # 储能末态微小激励，防止过度透支

    bounds = []
    for _ in range(T): bounds.append((0, None))                    # E_grid >= 0
    for _ in range(T): bounds.append((0, E_RATE_MAX))          # 0 <= E_ch <= 833.33 kWh
    for _ in range(T): bounds.append((0, E_RATE_MAX))          # 0 <= E_dis <= 833.33 kWh
    for _ in range(T): bounds.append((S_MIN + 300, S_MAX)) #  保留 300kWh 安全防御死区

    # 不等式约束: 供电量不可低于负荷预测值
    A_ub = np.zeros((T, 4 * T))
    b_ub = np.zeros(T)
    for t in range(T):
        A_ub[t, t] = -1.0
        A_ub[t, T + t] = 1.0
        A_ub[t, 2 * T + t] = -1.0
        b_ub[t] = (pv_pred[t] - load_pred[t]) * DELTA_T

    # 等式约束: 储能状态转移方程
    A_eq = np.zeros((T, 4 * T))
    b_eq = np.zeros(T)
    for t in range(T):
        A_eq[t, 3 * T + t] = 1.0
        if t > 0:
            A_eq[t, 3 * T + t - 1] = -1.0
        else:
            b_eq[t] = S_initial
        A_eq[t, T + t] = -ETA_CH
        A_eq[t, 2 * T + t] = 1.0 / ETA_DIS

    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
method='highs')
    if res.success:
        return res.x[:T]
    else:
        return np.maximum(0, (load_pred - pv_pred) * DELTA_T)

# ------------------------------------------------------------
```

```
# 5. 日内能量管理系统 (EMS) 动态闭环仿真
# ------------------------------------------------------------
def simulate_realtime_ems(price, E_grid_plan, load_real_day, pv_real_day, S_start):
    S_curr = S_start
    ch_record = np.zeros(T)
    dis_record = np.zeros(T)
    em_record = np.zeros(T)
    soc_record = np.zeros(T + 1)
    soc_record[0] = S_curr

    for t in range(T):
        e_load = load_real_day[t] * DELTA_T
        e_pv = pv_real_day[t] * DELTA_T
        e_plan = E_grid_plan[t]
        net_gap = e_load - (e_plan + e_pv)

        if net_gap <= 0:
            surplus = -net_gap
            max_ch_allow = min(E_RATE_MAX, (S_MAX - S_curr) / ETA_CH)
            actual_ch = min(surplus, max_ch_allow)

            ch_record[t] = actual_ch
            dis_record[t] = 0.0
            em_record[t] = 0.0
            S_curr += actual_ch * ETA_CH
        else:
            deficit = net_gap
            max_dis_allow = min(E_RATE_MAX, (S_curr - S_MIN) * ETA_DIS)
            actual_dis = min(deficit, max_dis_allow)

            ch_record[t] = 0.0
            dis_record[t] = actual_dis
            S_curr -= actual_dis / ETA_DIS

            unmet = deficit - actual_dis
            em_record[t] = unmet if unmet > 1e-4 else 0.0

        soc_record[t + 1] = S_curr

    return ch_record, dis_record, em_record, soc_record

# ------------------------------------------------------------
```

```
# 6. 全年滚动主循环
# ------------------------------------------------------------
def run_simulation(date_range, price_daily, load_all, pv_all, buffer_factor=1.03):
    days_total = len(date_range)
    all_grid_plan = np.zeros((days_total, T))
    all_ch = np.zeros((days_total, T))
    all_dis = np.zeros((days_total, T))
    all_em = np.zeros((days_total, T))
    all_soc = np.zeros((days_total, T + 1))

    current_S = S_INIT
    for d in range(days_total):
        l_pred, p_pred = predict_day_ahead(load_all, pv_all, d, buffer_factor=buffer_factor)
        grid_plan = solve_day_ahead_plan(price_daily, l_pred, p_pred, current_S)
        ch, dis, em, soc = simulate_realtime_ems(price_daily, grid_plan, load_all[d], pv_all[d],
current_S)

        all_grid_plan[d] = grid_plan
        all_ch[d] = ch
        all_dis[d] = dis
        all_em[d] = em
        all_soc[d] = soc
        current_S = soc[-1]

    return all_grid_plan, all_ch, all_dis, all_em, all_soc

# ------------------------------------------------------------
# 7. 方案 A：灵敏度分析与报童最优分位数权衡计算
# ------------------------------------------------------------
def run_sensitivity_tradeoff(date_range, price_daily, load_all, pv_all):
    print("\n>>> 正在运行方案 A：安全裕度与惩罚倍率敏感度权衡推演...")
    buffer_factors = np.linspace(0.98, 1.08, 6)
    buffer_results = []

    # 选取代表性时段进行快速精准权衡评估 (2 月至 12 月各月第 15 天)
    sample_day_indices = [
        np.where(date_range == f"2025-{m:02d}-15")[0][0]
        for m in range(2, 13)
    ]

    for bf in buffer_factors:
        tot_plan_cost = 0.0
```

```
            tot_em_energy = 0.0
            tot_em_cost = 0.0
            current_S = S_INIT

            for d_idx in sample_day_indices:
                l_pred, p_pred = predict_day_ahead(load_all, pv_all, d_idx, buffer_factor=bf)
                plan = solve_day_ahead_plan(price_daily, l_pred, p_pred, current_S)
                _, _, em, soc = simulate_realtime_ems(price_daily, plan, load_all[d_idx],
pv_all[d_idx], current_S)

                tot_plan_cost += np.sum(plan * price_daily)
                tot_em_energy += np.sum(em)
                tot_em_cost += np.sum(em * price_daily * PENALTY_RATIO)
                current_S = soc[-1]

            buffer_results.append({
                "buffer": bf,
                "plan_cost": tot_plan_cost,
                "em_energy": tot_em_energy,
                "em_cost": tot_em_cost,
                "total_cost": tot_plan_cost + tot_em_cost
            })

        df_buffer = pd.DataFrame(buffer_results)

        # 扫描紧急惩罚倍率 k 在基准策略下的影响
        k_values = np.linspace(1, 10, 10)
        k_costs = []
        base_plan_cost = buffer_results[2]["plan_cost"]
        base_em_energy = buffer_results[2]["em_energy"]
        base_base_em_cost = buffer_results[2]["em_cost"] / PENALTY_RATIO

        for k in k_values:
            k_costs.append(base_plan_cost + base_base_em_cost * k)

        return df_buffer, k_values, k_costs
# ------------------------------------------------------------
# 9. 赛题 Excel 导出规范与指定表格打印
# ------------------------------------------------------------
def export_and_print_tables(date_range, price_daily, all_grid_plan, all_ch, all_dis, all_em, all_soc):
        mask = (date_range >= "2025-02-01") & (date_range <= "2025-12-31")
        eval_dates_str = date_range[mask].strftime("%Y/%m/%d")
```

```
sub_plan = all_grid_plan[mask]
sub_ch = all_ch[mask]
sub_dis = all_dis[mask]
sub_em = all_em[mask]
sub_soc = all_soc[mask]

# Sheet 1: 计划购电量 (包含 144 个间隔时段以及全天购电量、购电费)
df_sheet1 = pd.DataFrame(sub_plan, index=eval_dates_str, columns=TIME_LABELS)
df_sheet1.index.name = "日期"
df_sheet1["全天购电量"] = np.sum(sub_plan, axis=1).round(4)
df_sheet1["全天购电费"] = np.dot(sub_plan, price_daily).round(4)

# Sheet 2: 充放电量
periods = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "20:00-24:00"]
ch_blocks = [sub_ch[:, i*24:(i+1)*24].sum(axis=1).round(4) for i in range(6)]
dis_blocks = [sub_dis[:, i*24:(i+1)*24].sum(axis=1).round(4) for i in range(6)]

sheet2_data = {}
for i, p in enumerate(periods):
    sheet2_data[f"{p} 充电量"] = ch_blocks[i]
    sheet2_data[f"{p} 放电量"] = dis_blocks[i]
sheet2_data["0:00 储电量"] = sub_soc[:, 0].round(4)
sheet2_data["24:00 储电量"] = sub_soc[:, -1].round(4)
df_sheet2 = pd.DataFrame(sheet2_data, index=eval_dates_str)
df_sheet2.index.name = "日期"

# Sheet 3: 紧急购电量
em_rows = []
for d_idx, d_str in enumerate(eval_dates_str):
    em_day = sub_em[d_idx]
    t = 0
    while t < T:
        if em_day[t] > 1e-4:
            start_t = t
            tot_em = 0.0
            while t < T and em_day[t] > 1e-4:
                tot_em += em_day[t]
                t += 1
            end_t = t
            start_str = f"{(start_t*10)//60:02d}:{(start_t*10)%60:02d}"
            end_str = f"{(end_t*10)//60:02d}:{(end_t*10)%60:02d}"
```

```
                em_rows.append({
                    "日期": d_str,
                    "紧急购电时间段": f"{start_str}-{end_str}",
                    "紧急购电量": round(tot_em, 4)
                })
            else:
                t += 1

    df_sheet3 = pd.DataFrame(em_rows)
    if df_sheet3.empty:
        df_sheet3 = pd.DataFrame(columns=["日期", "紧急购电时间段", "紧急购电量"])

    with pd.ExcelWriter("result2.xlsx", engine="openpyxl") as writer:
        df_sheet1.to_excel(writer, sheet_name="计划购电量")
        df_sheet2.to_excel(writer, sheet_name="充放电量")
        df_sheet3.to_excel(writer, sheet_name="紧急购电量", index=False)

    print(">>> 成果文件已导出至 'result2.xlsx' (已包含全天购电量与全天购电费)。")

def display_specified_date_tables(date_range, price_daily, all_grid_plan, all_ch, all_dis, all_em, all_soc):
    target_dates = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]

    print("\n" + "="*80)
    print("【表 1 格式】微网在指定日期的计划购电量及全天的购电量和购电费 (单位: kWh / 元)")
    print("="*80)
    target_slots = [
        ("10:00-10:10", 60), ("12:00-12:10", 72), ("14:00-14:10", 84),
        ("16:00-16:10", 96), ("18:00-18:10", 108), ("20:00-20:10", 120)
    ]

    col_headers = ["时间段 / 统计指标"] + target_dates
    row_list = []
    for slot_name, slot_idx in target_slots:
        row = [slot_name]
        for d in target_dates:
            d_idx = np.where(date_range == d)[0][0]
            row.append(f"{all_grid_plan[d_idx, slot_idx]:.4f}")
        row_list.append(row)

    # 全天计划购电量与费用
```

```
    tot_e_row = ["全天计划购电量"]
    tot_c_row = ["全天计划购电费"]
    for d in target_dates:
        d_idx = np.where(date_range == d)[0][0]
        tot_e = np.sum(all_grid_plan[d_idx])
        tot_c = np.sum(all_grid_plan[d_idx] * price_daily)
        tot_e_row.append(f"{tot_e:.4f}")
        tot_c_row.append(f"{tot_c:.4f}")
    row_list.append(tot_e_row)
    row_list.append(tot_c_row)

    print(pd.DataFrame(row_list, columns=col_headers).to_markdown(index=False))

    print("\n" + "="*80)
    print("【表 2 格式】储能设备在指定日期的充放电量及始末储电量统计 (单位: kWh)")
    print("="*80)
    periods = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "20:00-24:00"]
    for d in target_dates:
        d_idx = np.where(date_range == d)[0][0]
        print(f"\n>>> 日期: {d}")
        table2_records = []
        for p_idx, p_name in enumerate(periods):
            ch_val = np.sum(all_ch[d_idx, p_idx*24:(p_idx+1)*24])
            dis_val = np.sum(all_dis[d_idx, p_idx*24:(p_idx+1)*24])
            table2_records.append({"时间段": p_name, "充电量": f"{ch_val:.4f}", "放电量":
f"{dis_val:.4f}"})
        print(pd.DataFrame(table2_records).to_markdown(index=False))
        print(f"0:00 储电量: {all_soc[d_idx, 0]:.4f} kWh | 24:00 储电量: {all_soc[d_idx,
-1]:.4f} kWh")

    print("\n" + "="*80)
    print("【表 3 格式】微网在指定日期的紧急购电量统计表")
    print("="*80)
    em_summary = []
    for d in target_dates:
        d_idx = np.where(date_range == d)[0][0]
        em_day = all_em[d_idx]
        t = 0
        found = False
        while t < T:
            if em_day[t] > 1e-4:
                found = True
```

```
                    start_t = t
                    tot_val = 0.0
                    while t < T and em_day[t] > 1e-4:
                        tot_val += em_day[t]
                        t += 1
                    end_t = t
                    s_str = f"{(start_t*10)//60:02d}:{(start_t*10)%60:02d}"
                    e_str = f"{(end_t*10)//60:02d}:{(end_t*10)%60:02d}"
                    em_summary.append({"日期": d, "紧急购电时间段": f"{s_str}-{e_str}", "紧急购电量 (kWh)": f"{tot_val:.4f}"})
                else:
                    t += 1
            if not found:
                em_summary.append({"日期": d, "紧急购电时间段": "— (无紧急购电)", "紧急购电量 (kWh)": "0.0000"})
        print(pd.DataFrame(em_summary).to_markdown(index=False))

# ------------------------------------------------------------
# 10. 程序主入口
# ------------------------------------------------------------
if __name__ == "__main__":
        date_range, price_daily, load_all, pv_all = load_contest_data()
        all_grid_plan, all_ch, all_dis, all_em, all_soc = run_simulation(date_range, price_daily, load_all, pv_all)
        export_and_print_tables(date_range, price_daily, all_grid_plan, all_ch, all_dis, all_em, all_soc)
        display_specified_date_tables(date_range, price_daily, all_grid_plan, all_ch, all_dis, all_em, all_soc)
```

问题 3 代码：

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import linprog
import warnings
warnings.filterwarnings('ignore')

T_STEPS = 144                    # 每日 144 个 10 分钟时段
DELTA_T = 10.0 / 60.0            # 步长 1/6 小时
C_CAP = 12000.0                  # 储能额定容量 (kWh)
```

```
P_MAX = 5000.0                    # 最大充放电功率 (kW)
E_RATE_MAX = P_MAX * DELTA_T    # 833.3333 kWh
S_MIN = 1200.0                    # 储能容量下限 (kWh)
S_MAX = 10800.0                   # 储能容量上限 (kWh)
ETA_CH = 0.90                     # 充电效率
ETA_DIS = 0.90                    # 放电效率
GAMMA_END = 0.01                  # 日末储电量软引导系数 (元/kWh)

KEY_DATES = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']

# ---------------------------------------------------------------------
# 1. 数据读取与预处理模块
# ---------------------------------------------------------------------
def load_and_preprocess_data():
    print("正在读取附件 1、附件 2 与附件 3 时序数据...")

    # 1. 附件 1: 电价 (144 时段)
    df_att1 = pd.read_excel('附件 1.xlsx')
    price_profile = df_att1['电价'].values.astype(float)

    # 2. 附件 2: 全年小区实际负载与光伏实际功率 (365 天 x 144 列)
    df_load_real = pd.read_excel('附件 2.xlsx', sheet_name='小区负载')
    df_pv_real = pd.read_excel('附件 2.xlsx', sheet_name='光伏发电实际功率')

    df_load_real['日期\\时间'] = pd.to_datetime(df_load_real['日期\\时间']).dt.strftime('%Y-%m-%d')
    df_pv_real['日期\\时间'] = pd.to_datetime(df_pv_real['日期\\时间']).dt.strftime('%Y-%m-%d')

    # 提取 144 个 10 分钟列 (忽略首列'日期\时间')
    load_cols = df_load_real.columns[1:145]
    pv_cols = df_pv_real.columns[1:145]

    dict_load_real = {row['日期\\时间']: row[load_cols].values.astype(float) for _, row in df_load_real.iterrows()}
    dict_pv_real = {row['日期\\时间']: row[pv_cols].values.astype(float) for _, row in df_pv_real.iterrows()}

    # 3. 附件 3: 滚动光伏预报数据 (4 次/天 x 24 小时整点)
    df_att3 = pd.read_excel('附件 3.xlsx')
    df_att3['日期'] = df_att3['日期'].ffill()
    df_att3['日期'] = pd.to_datetime(df_att3['日期']).dt.strftime('%Y-%m-%d')
    df_att3['预报时刻'] = df_att3['预报时刻'].astype(str).str.strip()
```

```
    fc_cols = [f'预报{h}小时' for h in range(1, 25)]

    # 构建预报索引字典: dict_forecast[date][time_str] -> array(24,)
    dict_forecast = {}
    for _, row in df_att3.iterrows():
        d = row['日期']
        t_str = row['预报时刻']
        if d not in dict_forecast:
            dict_forecast[d] = {}
        dict_forecast[d][t_str] = row[fc_cols].values.astype(float)

    print("        数据清洗完成，时间序列与物理量对齐就绪。")
    return price_profile, dict_load_real, dict_pv_real, dict_forecast

# ---------------------------------------------------------------------------
# 2. 整点预报向 10 分钟离散时步的保形插值
# ---------------------------------------------------------------------------
def interpolate_forecast_to_10min(fc_24h, fc_hour_start):
    """
    将整点预报插值到 10 分钟区间 (当日剩余离散时步)
    fc_hour_start: 0, 6, 12, 18
    """
    # 整点对应的小时坐标 (以当日 0:00 为原点)
    fc_hours = np.arange(fc_hour_start + 1, fc_hour_start + 25)
    hours_10min = np.linspace(1/6, 24.0, 144)

    # 扩展包含当前起始时刻点 (出力设为 0 或邻接值)
    xp = np.insert(fc_hours, 0, fc_hour_start)
    yp = np.insert(fc_24h, 0, 0.0 if fc_hour_start in [0, 18] else fc_24h[0] * 0.8)

    # 线性保形插值并限制非负
    pv_10min = np.interp(hours_10min, xp, yp)
    return np.maximum(0.0, pv_10min)

# ---------------------------------------------------------------------------
# 3. 运筹优化核心算法: MPC 多阶段滚动求解引擎
# ---------------------------------------------------------------------------
def solve_day_ahead_plan(price, load_pred, pv_pred_0, s_init):
    """
    阶段 1: 0:00 求解全天 144 时步日前基准计划购电量 (LP)
    """
```

```
    T = T_STEPS
    # 决策变量向量: x = [E_grid(T), E_ch(T), E_dis(T), S(T)] -> 4*T 维
    c = np.zeros(4 * T)
    c[0:T] = price
    c[3*T + T - 1] = -GAMMA_END

    # 供需平衡不等式: -E_grid + E_ch - E_dis <= Epv - Eload
    E_load = load_pred * DELTA_T
    E_pv = pv_pred_0 * DELTA_T

    A_ub = np.zeros((T, 4 * T))
    b_ub = np.zeros(T)
    for t in range(T):
        A_ub[t, t] = -1.0
        A_ub[t, T + t] = 1.0
        A_ub[t, 2*T + t] = -1.0
        b_ub[t] = E_pv[t] - E_load[t]

    # 储能状态转移等式: S_t - S_{t-1} - eta_ch * E_ch + (1/eta_dis) * E_dis = 0
    A_eq = np.zeros((T, 4 * T))
    b_eq = np.zeros(T)
    for t in range(T):
        A_eq[t, 3*T + t] = 1.0
        if t > 0:
            A_eq[t, 3*T + t - 1] = -1.0
        else:
            b_eq[t] = s_init
        A_eq[t, T + t] = -ETA_CH
        A_eq[t, 2*T + t] = 1.0 / ETA_DIS

    # 变量上下界
    bounds = []
    for t in range(T): bounds.append((0.0, None))                  # E_grid
    for t in range(T): bounds.append((0.0, E_RATE_MAX))            # E_ch
    for t in range(T): bounds.append((0.0, E_RATE_MAX))            # E_dis
    for t in range(T): bounds.append((S_MIN, S_MAX))               # S_t

    res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
method='highs')
    if not res.success:
        raise ValueError("0:00 日前规划求解失败！")
```

```
    E_grid_plan = res.x[0:T]
    return E_grid_plan

def solve_intraday_adjustment(price, load_pred, pv_pred_tau, E_grid_plan, tau, s_tau):
    """
    阶段 2: 日内滚动调整 (tau 为决策时刻索引: 36(6:00), 72(12:00), 108(18:00))
    对剩余时段 t in [tau, 144) 求解增量调整量 E_grid_adj
    """
    T_rem = T_STEPS - tau
    if T_rem <= 0:
        return E_grid_plan.copy()

    sub_price = price[tau:T_STEPS]
    sub_E_plan = E_grid_plan[tau:T_STEPS]
    sub_E_load = load_pred[tau:T_STEPS] * DELTA_T
    sub_E_pv = pv_pred_tau[tau:T_STEPS] * DELTA_T

    # 变量: x = [delta_E_plus(T_rem), delta_E_minus(T_rem), E_ch(T_rem), E_dis(T_rem),
S(T_rem)] -> 5 * T_rem 维
    # 目标: 1.5*c*delta_E_plus - 0.5*c*delta_E_minus - gamma*S_end
    c = np.zeros(5 * T_rem)
    c[0:T_rem] = 1.5 * sub_price
    c[T_rem:2*T_rem] = -0.5 * sub_price
    c[4*T_rem + T_rem - 1] = -GAMMA_END

    # 约束 1: 供需平衡: - (E_plan + delta_plus - delta_minus) + E_ch - E_dis <= Epv - Eload
    # 即: - delta_plus + delta_minus + E_ch - E_dis <= Epv - Eload + E_plan
    # 约束 2: 购电量非负: - delta_plus + delta_minus <= E_plan
    A_ub = np.zeros((2 * T_rem, 5 * T_rem))
    b_ub = np.zeros(2 * T_rem)

    for i in range(T_rem):
        # 平衡
        A_ub[i, i] = -1.0
        A_ub[i, T_rem + i] = 1.0
        A_ub[i, 2*T_rem + i] = 1.0
        A_ub[i, 3*T_rem + i] = -1.0
        b_ub[i] = sub_E_pv[i] - sub_E_load[i] + sub_E_plan[i]
        # 非负购电
        A_ub[T_rem + i, i] = -1.0
        A_ub[T_rem + i, T_rem + i] = 1.0
        b_ub[T_rem + i] = sub_E_plan[i]
```

```
    # 约束 3: 储能状态转移
    A_eq = np.zeros((T_rem, 5 * T_rem))
    b_eq = np.zeros(T_rem)
    for i in range(T_rem):
        A_eq[i, 4*T_rem + i] = 1.0
        if i > 0:
            A_eq[i, 4*T_rem + i - 1] = -1.0
        else:
            b_eq[i] = s_tau
        A_eq[i, 2*T_rem + i] = -ETA_CH
        A_eq[i, 3*T_rem + i] = 1.0 / ETA_DIS

    bounds = []
    for i in range(T_rem): bounds.append((0.0, None))                # delta_plus
    for i in range(T_rem): bounds.append((0.0, None))                # delta_minus
    for i in range(T_rem): bounds.append((0.0, E_RATE_MAX))           # E_ch
    for i in range(T_rem): bounds.append((0.0, E_RATE_MAX))           # E_dis
    for i in range(T_rem): bounds.append((S_MIN, S_MAX))              # S_t

    res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
method='highs')
    if not res.success:
        # 若极度紧缩无法收敛，保持日前计划
        return E_grid_plan.copy()

    delta_plus = res.x[0:T_rem]
    delta_minus = res.x[T_rem:2*T_rem]

    E_grid_adj = E_grid_plan.copy()
    E_grid_adj[tau:T_STEPS] = sub_E_plan + delta_plus - delta_minus
    return E_grid_adj

# ------------------------------------------------------------------------
# 4. 全年滚动仿真与日内物理闭环执行 (EMS)
# ------------------------------------------------------------------------
def run_full_year_simulation(price, dict_load_real, dict_pv_real, dict_forecast):
    print("启动 2025 全年 365 天逐日多阶段滚动 MPC 闭环仿真...")

    dates = sorted(list(dict_load_real.keys()))
    s_current = 6000.0   # 初始储电量
```

```
# 结果容器
results_plan = {}
results_adj = {}
results_ems = {}

for day_idx, d in enumerate(dates):
    P_load_real = dict_load_real[d]
    P_pv_real = dict_pv_real[d]

    # 负荷预测: 采用前 7 天滑动均值作为前瞻期望，冷启动用当日加扰动
    if day_idx >= 7:
        recent_days = dates[day_idx-7:day_idx]
        P_load_pred = np.mean([dict_load_real[k] for k in recent_days], axis=0)
    else:
        P_load_pred = P_load_real.copy()

    # 获取光伏预报
    fc_dict_day = dict_forecast.get(d, {})
    fc_0 = fc_dict_day.get('0:00', np.zeros(24))
    fc_6 = fc_dict_day.get('6:00', fc_0)
    fc_12 = fc_dict_day.get('12:00', fc_6)
    fc_18 = fc_dict_day.get('18:00', fc_12)

    P_pv_pred_0 = interpolate_forecast_to_10min(fc_0, 0)
    P_pv_pred_6 = interpolate_forecast_to_10min(fc_6, 6)
    P_pv_pred_12 = interpolate_forecast_to_10min(fc_12, 12)
    P_pv_pred_18 = interpolate_forecast_to_10min(fc_18, 18)

    # 1. 0:00 制定日前基准计划
    E_plan = solve_day_ahead_plan(price, P_load_pred, P_pv_pred_0, s_current)

    # 2. 日内滚动执行与调整
    # 初始时段 (0:00 - 6:00, 时步 0~35) 跟踪 E_plan
    E_final_grid = E_plan.copy()
    s_sim = s_current

    # 逐时步执行并记录
    E_ch_real = np.zeros(T_STEPS)
    E_dis_real = np.zeros(T_STEPS)
    E_em_real = np.zeros(T_STEPS)
    soc_real = np.zeros(T_STEPS)
```

```
        for t in range(T_STEPS):
            # 决策时刻触发调整
            if t == 36:    # 06:00
                E_final_grid = solve_intraday_adjustment(price, P_load_pred, P_pv_pred_6,
E_final_grid, 36, s_sim)
            elif t == 72: # 12:00
                E_final_grid = solve_intraday_adjustment(price, P_load_pred, P_pv_pred_12,
E_final_grid, 72, s_sim)
            elif t == 108: # 18:00
                E_final_grid = solve_intraday_adjustment(price, P_load_pred, P_pv_pred_18,
E_final_grid, 108, s_sim)

            # 物理 EMS 状态机响应
            net_gap = P_load_real[t] * DELTA_T - (E_final_grid[t] + P_pv_real[t] * DELTA_T)

            if net_gap <= 0:
                # 绿电与购电富余 -> 充电
                surplus = -net_gap
                max_ch_allow = min(E_RATE_MAX, (S_MAX - s_sim) / ETA_CH)
                ch_val = min(surplus, max_ch_allow)
                E_ch_real[t] = ch_val
                E_dis_real[t] = 0.0
                E_em_real[t] = 0.0
                s_sim = s_sim + ETA_CH * ch_val
            else:
                # 能量匮乏 -> 储能放电保供
                max_dis_allow = min(E_RATE_MAX, (s_sim - S_MIN) * ETA_DIS)
                dis_val = min(net_gap, max_dis_allow)
                E_dis_real[t] = dis_val
                E_ch_real[t] = 0.0
                shortfall = net_gap - dis_val
                if shortfall > 1e-4:
                    E_em_real[t] = shortfall
                else:
                    E_em_real[t] = 0.0
                s_sim = s_sim - (1.0 / ETA_DIS) * dis_val

            soc_real[t] = s_sim

        # 跨日状态连续转移
        s_current = s_sim
```

```
        # 存储结果
        results_plan[d] = E_plan
        results_adj[d] = E_final_grid
        results_ems[d] = {
            'E_ch': E_ch_real, 'E_dis': E_dis_real, 'E_em': E_em_real, 'soc': soc_real,
            's_start': soc_real[0], 's_end': s_sim
        }

    print("          全年闭环仿真收敛，各日滚动策略计算完成。")
    return results_plan, results_adj, results_ems

# ---------------------------------------------------------------------
# 5. 控制台规范表格打印 (表 1、表 2、表 3)
# ---------------------------------------------------------------------
def print_submission_tables(price, results_plan, results_adj, results_ems):
    print("\n" + "="*95)
    print("                                【表 1 微网在指定日期的计划与调整购电结果】")
    print("="*95)

    spec_indices = [60, 72, 84, 96, 108, 120]   # 10:00, 12:00, 14:00, 16:00, 18:00, 20:00 (10 分钟段)
    spec_labels = ['10:00-10:10', '12:00-12:10', '14:00-14:10', '16:00-16:10', '18:00-18:10', '20:00-20:10']

    headers = ["时间段 / 统计指标"] + [f"{d}" for d in KEY_DATES]
    row_format = "{:<20}" + "{:>18}" * len(KEY_DATES)
    print(row_format.format(*headers))
    print("-" * 95)

    for idx, lbl in zip(spec_indices, spec_labels):
        vals = [f"{results_adj[d][idx]:.4f} kWh" for d in KEY_DATES]
        print(row_format.format(lbl, *vals))

    print("-" * 95)
    # 全天指标
    tot_plans = [f"{np.sum(results_plan[d]):.4f} kWh" for d in KEY_DATES]
    tot_adjs  = [f"{np.sum(results_adj[d]):.4f} kWh" for d in KEY_DATES]

    # 费用计算 (原价计划 + 1.5 倍增量 - 0.5 倍减量 + 5 倍紧急)
    tot_costs = []
    for d in KEY_DATES:
        E_p = results_plan[d]
```

```
            E_a = results_adj[d]
            E_em = results_ems[d]['E_em']
            delta = E_a - E_p
            d_plus = np.maximum(0.0, delta)
            d_minus = np.maximum(0.0, -delta)
            cost = np.sum(price * E_p + 1.5 * price * d_plus - 0.5 * price * d_minus + 5.0 * price *
E_em)
            tot_costs.append(f"{cost:.4f} 元")

        print(row_format.format("全天计划购电量", *tot_plans))
        print(row_format.format("全天最终购电量", *tot_adjs))
        print(row_format.format("全天综合总电费", *tot_costs))
        print("="*95)

        print("\n" + "="*95)
        print("                                【表 2 储能设备在指定日期的充放电量及储电量统计 (kWh)】
")
        print("="*95)
        t_ranges = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
        r_labels = ['0:00-4:00', '4:00-8:00', '8:00-12:00', '12:00-16:00', '16:00-20:00', '20:00-24:00']

        for d in KEY_DATES:
            ems = results_ems[d]
            ch_blocks = [f"{np.sum(ems['E_ch'][st:ed]):.4f}" for st, ed in t_ranges]
            dis_blocks = [f"{np.sum(ems['E_dis'][st:ed]):.4f}" for st, ed in t_ranges]

            print(f"\n>>> 日期: {d} | 0:00 储电量: {ems['soc'][0]:.4f} kWh | 24:00 储电量:
{ems['s_end']:.4f} kWh")
            sub_head = "{:<12}" + "{:>13}" * 6
            print(sub_head.format("时段", *r_labels))
            print(sub_head.format("充电量", *ch_blocks))
            print(sub_head.format("放电量", *dis_blocks))
        print("="*95)

        print("\n" + "="*95)
        print("                                【表 3 微网在指定日期的紧急购电量统计】")
        print("="*95)
        print("{:<15} {:<25} {:<20} {:<30}".format("指定日期", "紧急购电时间段", "紧急购电量
(kWh)", "系统响应与机理说明"))
        print("-" * 95)

        for d in KEY_DATES:
```

```
            E_em = results_ems[d]['E_em']
            total_em = np.sum(E_em)
            if total_em < 1e-3:
                print("{:<15} {:<25} {:<20} {:<30}".format(
                    d, "—  (全天无紧急缺电)", "0.0000", "日内 MPC 动态重调+储能保供完全覆盖
"
                ))
            else:
                # 提取连续缺电区间
                in_ev = False
                st_t = 0
                for t in range(T_STEPS):
                    if E_em[t] > 1e-3 and not in_ev:
                        in_ev = True
                        st_t = t
                    elif E_em[t] <= 1e-3 and in_ev:
                        in_ev = False
                        h1, m1 = divmod(st_t * 10, 60)
                        h2, m2 = divmod(t * 10, 60)
                        seg_lbl = f"{h1:02d}:{m1:02d}-{h2:02d}:{m2:02d}"
                        seg_val = np.sum(E_em[st_t:t])
                        print("{:<15} {:<25} {:<20.4f} {:<30}".format(
                            d, seg_lbl, seg_val, "突发负荷脉冲超出储能放电功率上限"
                        ))
        print("="*95 + "\n")

# ------------------------------------------------------------------------
# 6. 导出至 result3.xlsx (严格遵循附件 5 模板四工作表结构)
# ------------------------------------------------------------------------
def export_result3_excel(price, results_plan, results_adj, results_ems):
    print("正在将策略数据按格式导出写入 result3.xlsx...")

    eval_dates = [d for d in sorted(list(results_plan.keys())) if d >= '2025-02-01']

    time_headers = [f"{int(t*10//60):02d}:{int(t*10%60):02d}-{int((t+1)*10//60):02d}:{int((t+1)*10%60):02d}" for t in range(T_STEPS)]

    # 1. 计划购电量
    data_p = []
    for d in eval_dates:
        row = [d] + list(np.round(results_plan[d], 4))
```

```
        tot_q = np.sum(results_plan[d])
        tot_f = np.sum(results_plan[d] * price)
        row.extend([round(tot_q, 4), round(tot_f, 4)])
        data_p.append(row)
    df_plan = pd.DataFrame(data_p, columns=['日期\\时间'] + time_headers + ['全天购电量', '全天购电费'])

    # 2. 调整购电量
    data_a = []
    for d in eval_dates:
        row = [d] + list(np.round(results_adj[d], 4))
        tot_q = np.sum(results_adj[d])
        tot_f = np.sum(results_adj[d] * price)
        row.extend([round(tot_q, 4), round(tot_f, 4)])
        data_a.append(row)
    df_adj = pd.DataFrame(data_a, columns=['日期\\时间'] + time_headers + ['全天购电量', '全天购电费'])

    # 3. 充放电量 (按日内 144 时步记录储能细粒度运转轨迹)
    data_bat = []
    for d in eval_dates:
        ems = results_ems[d]
        for t in range(T_STEPS):
            t_lbl = time_headers[t]
            t_point = f"{int(t*10//60):02d}:{int(t*10%60):02d}"
            data_bat.append([d, t_lbl, round(ems['E_ch'][t], 4), round(ems['E_dis'][t], 4), t_point, round(ems['soc'][t], 4)])
    df_battery = pd.DataFrame(data_bat, columns=['日期', '时间段', '充电量', '放电量', '时刻', '储电量'])

    # 4. 紧急购电量
    data_em = []
    for d in eval_dates:
        E_em = results_ems[d]['E_em']
        in_ev = False
        st_t = 0
        for t in range(T_STEPS):
            if E_em[t] > 1e-4 and not in_ev:
                in_ev = True
                st_t = t
            elif E_em[t] <= 1e-4 and in_ev:
                in_ev = False
```

```
                    seg_lbl                                                                                    =
f"{int(st_t*10//60):02d}:{int(st_t*10%60):02d}-{int(t*10//60):02d}:{int(t*10%60):02d}"
                    seg_val = np.sum(E_em[st_t:t])
                    data_em.append([d, seg_lbl, round(seg_val, 4)])
            if in_ev:
                seg_lbl = f"{int(st_t*10//60):02d}:{int(st_t*10%60):02d}-24:00"
                seg_val = np.sum(E_em[st_t:])
                data_em.append([d, seg_lbl, round(seg_val, 4)])

        if len(data_em) == 0:
            data_em.append(['2025/2/1', '—  (无紧急购电)', 0.0])
        df_emergency = pd.DataFrame(data_em, columns=['日期', '购电时间段', '购电量'])

        with pd.ExcelWriter('result3.xlsx', engine='openpyxl') as writer:
            df_plan.to_excel(writer, sheet_name='计划购电量', index=False)
            df_adj.to_excel(writer, sheet_name='调整购电量', index=False)
            df_battery.to_excel(writer, sheet_name='充放电量', index=False)
            df_emergency.to_excel(writer, sheet_name='紧急购电量', index=False)

        print("          result3.xlsx  写入完成，完全吻合附件 5 机检规范。")

# ------------------------------------------------------------------------
#  主执行入口
# ------------------------------------------------------------------------
if __name__ == '__main__':
        print("="*95)
        print("        全国大学生数学建模竞赛  问题 3:  微网日内多阶段滚动调控求解程序")
        print("="*95)

        # 1.  载入真实附件数据
        price, dict_load_real, dict_pv_real, dict_forecast = load_and_preprocess_data()

        # 2.  全年逐日  MPC  滚动求解
        results_plan, results_adj, results_ems = run_full_year_simulation(
            price, dict_load_real, dict_pv_real, dict_forecast
        )

        # 3.  输出论文表格  (表 1,  表 2,  表 3)
        print_submission_tables(price, results_plan, results_adj, results_ems)

        # 4.  生成标准提交文件  result3.xlsx
        export_result3_excel(price, results_plan, results_adj, results_ems)
```

```
print("程序全部顺利执行完毕！")
```

问题 4 代码：

```
import os
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.interpolate import PchipInterpolator
import warnings
warnings.filterwarnings('ignore')

import matplotlib.pyplot as plt
# ========================================
# 1. 物理参数与运行常量
# ========================================
DT = 1.0 / 6.0                    # 10 分钟离散步长 (h)
STEPS_PER_DAY = 144               # 每天 144 个时步
BATTERY_CAP = 12000.0             # 电池容量 (kWh)
P_MAX = 5000.0                    # PCS 最大充放电功率 (kW)
E_PCS_MAX = P_MAX * DT            # 单步最大充放电量: 833.3333 kWh
ETA_CH = 0.90                     # 充电效率
ETA_DIS = 0.90                    # 放电效率
S_MIN = 1200.0                    # 安全储电下限 (kWh)
S_MAX = 10800.0                   # 安全储电上限 (kWh)
S_INIT_TOTAL = 6000.0             # 2025-01-01 0:00 初始电量 (kWh)
GAMMA_END = 0.01                  # 日末储电量残值激励系数 (元/kWh)

# 表1 指定采样的6个10分钟时段及对应时步索引 (10:00, 12:00, 14:00, 16:00, 18:00, 20:00 后10 分钟)
TABLE1_KEYS = ['10:00-10:10', '12:00-12:10', '14:00-14:10', '16:00-16:10', '18:00-18:10', '20:00-20:10']
TABLE1_INDICES = [60, 72, 84, 96, 108, 120]

# 表 2 指定充放电的 6 个 4 小时时段区间
TABLE2_BLOCKS = [
    ('0:00-4:00',    0, 24),
    ('4:00-8:00',   24, 48),
    ('8:00-12:00', 48, 72),
    ('12:00-16:00', 72, 96),
```

```
    ('16:00-20:00', 96, 120),
    ('20:00-24:00', 120, 144)
]

TYPICAL_DATES = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']

# ========================================
# 2. 数据读取与预报单调保形插值处理
# ========================================
def load_data():
    print(">>> 正在加载 附件 2.xlsx、附件 3.xlsx、附件 4.xlsx 原始数据...")
    df_load_raw = pd.read_excel('附件 2.xlsx', sheet_name='小区负载')
    df_pv_raw = pd.read_excel('附件 2.xlsx', sheet_name='光伏发电实际功率')
    df_price_raw = pd.read_excel('附件 4.xlsx', sheet_name='Sheet1')
    df_fc_raw = pd.read_excel('附件 3.xlsx', sheet_name='Sheet1')

    # 规范化日期格式为 YYYY-MM-DD
    df_load_raw['date_str'] = pd.to_datetime(df_load_raw.iloc[:, 0]).dt.strftime('%Y-%m-%d')
    df_pv_raw['date_str'] = pd.to_datetime(df_pv_raw.iloc[:, 0]).dt.strftime('%Y-%m-%d')
    df_price_raw['date_str'] = pd.to_datetime(df_price_raw.iloc[:, 0]).dt.strftime('%Y-%m-%d')
    df_fc_raw['date_str'] = pd.to_datetime(df_fc_raw['日期']).dt.strftime('%Y-%m-%d')

    date_list = sorted(list(df_load_raw['date_str'].unique()))

    P_load_dict, P_pv_dict, Price_dict = {}, {}, {}
    for d in date_list:
        P_load_dict[d] = df_load_raw[df_load_raw['date_str'] == d].iloc[0,
1:145].values.astype(float)
        P_pv_dict[d] = df_pv_raw[df_pv_raw['date_str'] == d].iloc[0, 1:145].values.astype(float)
        Price_dict[d] = df_price_raw[df_price_raw['date_str'] == d].iloc[0,
1:145].values.astype(float)

    # 使用 PCHIP 单调保形插值将 24 小时整点预报平滑展开至 144 个 10 分钟离散步
    PV_forecast_dict = {}
    for d in date_list:
        PV_forecast_dict[d] = {}
        sub_fc = df_fc_raw[df_fc_raw['date_str'] == d]
        for _, row in sub_fc.iterrows():
            t_str = str(row['预报时刻']).strip()
            y_24h = [float(row[f'预报{h}小时']) for h in range(1, 25)]
            x_pts = np.arange(1, 25)
            # 引入起点 0:00 (功率为 0) 形成平滑连续边界
```

```
            x_pts = np.insert(x_pts, 0, 0)
            y_24h = np.insert(y_24h, 0, 0.0)
            pchip = PchipInterpolator(x_pts, y_24h, extrapolate=True)
            x_interp = np.linspace(0, 24, 144)
            y_interp = np.maximum(0.0, pchip(x_interp))
            PV_forecast_dict[d][t_str] = y_interp

    return date_list, P_load_dict, P_pv_dict, Price_dict, PV_forecast_dict

# ========================================
# 3. 线性规划核心求解器 (日前与日内 MPC)
# ========================================
def solve_day_ahead_lp(c_price, E_load_fc, E_pv_fc, S_init, S_min_arr):
    """日前规划层 LP：求解基准计划购电量与计划充放电策略"""
    N = STEPS_PER_DAY
    c_obj = np.zeros(4 * N)
    c_obj[0:N] = c_price
    c_obj[4 * N - 1] = -GAMMA_END

    # 能量平衡约束: -E_grid + E_ch - E_dis <= E_pv - E_load
    A_ub = np.zeros((N, 4 * N))
    b_ub = np.zeros(N)
    for t in range(N):
        A_ub[t, t] = -1.0
        A_ub[t, N + t] = 1.0
        A_ub[t, 2 * N + t] = -1.0
        b_ub[t] = E_pv_fc[t] - E_load_fc[t]

    # 储能状态方程: S_t - S_{t-1} - eta_ch*E_ch,t + (1/eta_dis)*E_dis,t = 0
    A_eq = np.zeros((N, 4 * N))
    b_eq = np.zeros(N)
    for t in range(N):
        A_eq[t, 3 * N + t] = 1.0
        A_eq[t, N + t] = -ETA_CH
        A_eq[t, 2 * N + t] = 1.0 / ETA_DIS
        if t == 0:
            b_eq[t] = S_init
        else:
            A_eq[t, 3 * N + t - 1] = -1.0
            b_eq[t] = 0.0

    bounds = []
```

```
    for _ in range(N): bounds.append((0.0, None))
    for _ in range(N): bounds.append((0.0, E_PCS_MAX))
    for _ in range(N): bounds.append((0.0, E_PCS_MAX))
    for t in range(N): bounds.append((S_min_arr[t], S_MAX))

    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
    if res.success:
        return res.x[0:N]
    return np.maximum(0.0, E_load_fc - E_pv_fc)

def solve_intraday_mpc_lp(tau, c_price_rem, E_plan_rem, E_load_fc_rem, E_pv_fc_rem, S_real_tau):
    """日内滚动 MPC 层 LP：在非对称惩罚(1.5x 增购 / -0.5x 核减返还)下滚动优化剩余时段购电量"""
    K = len(c_price_rem)
    c_obj = np.zeros(5 * K)
    c_obj[0:K] = 1.5 * c_price_rem
    c_obj[K:2 * K] = -0.5 * c_price_rem
    c_obj[5 * K - 1] = -GAMMA_END

    A_ub = np.zeros((2 * K, 5 * K))
    b_ub = np.zeros(2 * K)
    for k in range(K):
        A_ub[k, k] = -1.0
        A_ub[k, K + k] = 1.0
        A_ub[k, 2 * K + k] = 1.0
        A_ub[k, 3 * K + k] = -1.0
        b_ub[k] = E_plan_rem[k] + E_pv_fc_rem[k] - E_load_fc_rem[k]

        A_ub[K + k, k] = -1.0
        A_ub[K + k, K + k] = 1.0
        b_ub[K + k] = E_plan_rem[k]

    A_eq = np.zeros((K, 5 * K))
    b_eq = np.zeros(K)
    for k in range(K):
        A_eq[k, 4 * K + k] = 1.0
        A_eq[k, 2 * K + k] = -ETA_CH
        A_eq[k, 3 * K + k] = 1.0 / ETA_DIS
        if k == 0:
            b_eq[k] = S_real_tau
```

```
            else:
                A_eq[k, 4 * K + k - 1] = -1.0
                b_eq[k] = 0.0

        bounds = []
        for _ in range(K): bounds.append((0.0, None))
        for _ in range(K): bounds.append((0.0, None))
        for _ in range(K): bounds.append((0.0, E_PCS_MAX))
        for _ in range(K): bounds.append((0.0, E_PCS_MAX))
        for _ in range(K): bounds.append((S_MIN, S_MAX))

        res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
method='highs')
        if res.success:
            d_plus = res.x[0:K]
            d_minus = res.x[K:2 * K]
            return E_plan_rem + d_plus - d_minus
        return E_plan_rem

# ==========================================
# 4. 日内 EMS 物理执行仿真状态机
# ==========================================
def simulate_realtime_ems(E_grid_exec, E_load_real, E_pv_real, S_prev):
        """实时闭环跟踪：储能响应不平衡量，不足部分触发 5 倍瞬时动态电价紧急购电"""
        N = len(E_grid_exec)
        E_ch_real = np.zeros(N)
        E_dis_real = np.zeros(N)
        E_em_real = np.zeros(N)
        S_real = np.zeros(N)

        s_curr = S_prev
        for t in range(N):
            gap = E_load_real[t] - (E_grid_exec[t] + E_pv_real[t])
            if gap <= 0.0:
                surplus = -gap
                max_ch = min(E_PCS_MAX, (S_MAX - s_curr) / ETA_CH)
                e_ch = min(surplus, max_ch)
                e_dis = 0.0
                e_em = 0.0
                s_curr = s_curr + ETA_CH * e_ch
            else:
                deficit = gap
```

```
            max_dis = min(E_PCS_MAX, (s_curr - S_MIN) * ETA_DIS)
            e_dis = min(deficit, max_dis)
            e_ch = 0.0
            e_em = deficit - e_dis
            s_curr = s_curr - e_dis / ETA_DIS

        E_ch_real[t] = e_ch
        E_dis_real[t] = e_dis
        E_em_real[t] = e_em
        S_real[t] = s_curr

    return E_ch_real, E_dis_real, E_em_real, S_real

# ========================================
# 5. 全年推演与求解流水线
# ========================================
def run_all_models(date_list, P_load_dict, P_pv_dict, Price_dict, PV_forecast_dict):
    results_4_2 = {}
    results_4_3 = {}
    S_curr_42 = S_INIT_TOTAL
    S_curr_43 = S_INIT_TOTAL

    print("\n>>> 开始全周期（2025.1.1 - 2025.12.31）长时序高保真推演...")
    for idx, d in enumerate(date_list):
        P_load_real = P_load_dict[d]
        P_pv_real = P_pv_dict[d]
        c_price = Price_dict[d]
        E_load_real = P_load_real * DT
        E_pv_real = P_pv_real * DT

        # 7 天滑动历史均值基准预测
        if idx >= 7:
            load_hist = [P_load_dict[date_list[idx - k]] for k in range(1, 8)]
            pv_hist = [P_pv_dict[date_list[idx - k]] for k in range(1, 8)]
            P_load_base = np.mean(load_hist, axis=0)
            P_pv_base = np.mean(pv_hist, axis=0)
        else:
            P_load_base = P_load_real.copy()
            P_pv_base = P_pv_real.copy()

        # ---------- 求解问题 4-2 (日前鲁棒分位数防御) ----------
        E_load_fc_42 = (1.03 * P_load_base) * DT
```

```
        E_pv_fc_42 = (0.95 * P_pv_base) * DT
        S_min_arr_42 = np.full(STEPS_PER_DAY, S_MIN)
        S_min_arr_42[36:] = S_MIN + 300.0    # 6:00 后施加安全防御死区

        plan_42 = solve_day_ahead_lp(c_price, E_load_fc_42, E_pv_fc_42, S_curr_42,
S_min_arr_42)
        ch_42, dis_42, em_42, S_42 = simulate_realtime_ems(plan_42, E_load_real, E_pv_real,
S_curr_42)
        s0_42 = S_curr_42
        S_curr_42 = S_42[-1]

        cost_plan_42 = np.sum(c_price * plan_42)
        cost_em_42 = np.sum(5.0 * c_price * em_42)
        cost_total_42 = cost_plan_42 + cost_em_42

        results_4_2[d] = {
            'plan': plan_42, 'ch': ch_42, 'dis': dis_42, 'em': em_42,
            'S': S_42, 's0': s0_42, 's24': S_42[-1], 'cost': cost_total_42,
            'plan_vol': np.sum(plan_42), 'plan_cost': cost_plan_42
        }

        # ---------- 求解问题 4-3 (日内多时序滚动 MPC 纠偏) ----------
        pv_fc_00 = PV_forecast_dict[d].get('0:00', P_pv_base) * DT
        E_load_fc_43 = P_load_base * DT
        S_min_arr_43 = np.full(STEPS_PER_DAY, S_MIN)

        plan_43 = solve_day_ahead_lp(c_price, E_load_fc_43, pv_fc_00, S_curr_43,
S_min_arr_43)
        exec_43 = plan_43.copy()

        s_mpc_curr = S_curr_43
        s0_43 = S_curr_43
        tau_cuts = [(0, 36, '0:00'), (36, 72, '6:00'), (72, 108, '12:00'), (108, 144, '18:00')]

        ch_43_full = np.zeros(STEPS_PER_DAY)
        dis_43_full = np.zeros(STEPS_PER_DAY)
        em_43_full = np.zeros(STEPS_PER_DAY)
        S_43_full = np.zeros(STEPS_PER_DAY)

        for t_st, t_ed, cast_k in tau_cuts:
            if t_st > 0:
                pv_rem = PV_forecast_dict[d].get(cast_k, P_pv_base)[t_st:] * DT
```

```
                load_rem = E_load_fc_43[t_st:]
                plan_rem = plan_43[t_st:]
                price_rem = c_price[t_st:]
                exec_43[t_st:] = solve_intraday_mpc_lp(t_st, price_rem, plan_rem, load_rem,
pv_rem, s_mpc_curr)

            c_seg, d_seg, e_seg, s_seg = simulate_realtime_ems(
                exec_43[t_st:t_ed], E_load_real[t_st:t_ed], E_pv_real[t_st:t_ed], s_mpc_curr
            )
            ch_43_full[t_st:t_ed] = c_seg
            dis_43_full[t_st:t_ed] = d_seg
            em_43_full[t_st:t_ed] = e_seg
            S_43_full[t_st:t_ed] = s_seg
            s_mpc_curr = s_seg[-1]

        S_curr_43 = S_43_full[-1]

        d_plus = np.maximum(0.0, exec_43 - plan_43)
        d_minus = np.maximum(0.0, plan_43 - exec_43)
        cost_plan_base = np.sum(c_price * plan_43)
        cost_adj_net = np.sum(1.5 * c_price * d_plus - 0.5 * c_price * d_minus)
        cost_em_43 = np.sum(5.0 * c_price * em_43_full)
        cost_total_43 = cost_plan_base + cost_adj_net + cost_em_43

        results_4_3[d] = {
            'plan': plan_43, 'adj': exec_43, 'ch': ch_43_full, 'dis': dis_43_full,
            'em': em_43_full, 'S': S_43_full, 's0': s0_43, 's24': S_43_full[-1],
            'cost': cost_total_43, 'exec_vol': np.sum(exec_43), 'plan_vol': np.sum(plan_43),
            'plan_cost': cost_plan_base
        }

    print(">>> 全年推演完成！")
    return results_4_2, results_4_3

# ==========================================
# 6. 控制台格式化输出：表 1、表 2、表 3 深度美化展示
# ==========================================
def display_table1(results, model_label, is_q3=False):
    print(f"\n{'='*98}")
    print(f"            【{model_label}】表 1　微网在指定时间段的购电量及全天的购电量和购电
费")
    print(f"{'='*98}")
```

```
    for d in TYPICAL_DATES:
        res = results[d]
        p_vec = res['adj'] if is_q3 else res['plan']
        vol_total = np.sum(p_vec)
        cost_total = res['cost']

        print(f"\n▶ 典型日期: {d}")
        print(f"+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+")
        print(f"| {'时间段':<16} | {'购电量(kWh)':>10} | {'时间段':<16} | {'购电量(kWh)':>10} | {'时间段':<16} | {'购电量(kWh)':>10} |")
        print(f"+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+")
        print(f"| {'10:00-10:10':<16} | {p_vec[60]:>10.4f} | {'12:00-12:10':<16} | {p_vec[72]:>10.4f} | {'14:00-14:10':<16} | {p_vec[84]:>10.4f} |")
        print(f"| {'16:00-16:10':<16} | {p_vec[96]:>10.4f} | {'18:00-18:10':<16} | {p_vec[108]:>10.4f} | {'20:00-20:10':<16} | {p_vec[120]:>10.4f} |")
        print(f"+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+")
        print(f"| {'全天购电量(kWh)':<16} | {vol_total:>10.4f} | {'全天购电费(元)':<16} | {cost_total:>10.4f} | {'-':<16} | {'-':>10} |")
        print(f"+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+{'-'*18}+{'-'*12}+")

    print(f"\n>>> 【{model_label}】表 1 4 个指定日期横向汇总一览表:")
    print(f"+{'-'*12}+" + "+".join([f"{'-'*13}" for _ in range(6)]) + f"+{'-'*15}+{'-'*14}+")
    headers = ["日期"] + TABLE1_KEYS + ["全天购电量(kWh)", "全天购电费(元)"]
    print(f"| {headers[0]:<10} | " + " | ".join([f"{h:>11}" for h in headers[1:7]]) + f" | {headers[7]:>13} | {headers[8]:>12} |")
    print(f"+{'-'*12}+" + "+".join([f"{'-'*13}" for _ in range(6)]) + f"+{'-'*15}+{'-'*14}+")
    for d in TYPICAL_DATES:
        res = results[d]
        p_vec = res['adj'] if is_q3 else res['plan']
        vals = [f"{p_vec[idx]:.4f}" for idx in TABLE1_INDICES]
        print(f"| {d:<10} | " + " | ".join([f"{v:>11}" for v in vals]) + f" | {np.sum(p_vec):>13.4f} | {res['cost']:>12.4f} |")
    print(f"+{'-'*12}+" + "+".join([f"{'-'*13}" for _ in range(6)]) + f"+{'-'*15}+{'-'*14}+")

def display_table2(results, model_label):
    print(f"\n{'='*98}")
    print(f"  【{model_label}】表 2  储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量 (单位: kWh)")
    print(f"{'='*98}")

    for d in TYPICAL_DATES:
```

```
        res = results[d]
        print(f"\n▶ 典型日期: {d}")
        print(f"+{'-'*14}+{'-'*13}+{'-'*13}+{'-'*14}+{'-'*13}+{'-'*13}+")
        print(f"| {'时间段':<12} | {'充电量(kWh)':>11} | {'放电量(kWh)':>11} | {'时间段':<12} | {'充电量(kWh)':>11} | {'放电量(kWh)':>11} |")
        print(f"+{'-'*14}+{'-'*13}+{'-'*13}+{'-'*14}+{'-'*13}+{'-'*13}+")

        ch1 = np.sum(res['ch'][0:24]); dis1 = np.sum(res['dis'][0:24])
        ch2 = np.sum(res['ch'][24:48]); dis2 = np.sum(res['dis'][24:48])
        print(f"| {'0:00-4:00':<12} | {ch1:>11.4f} | {dis1:>11.4f} | {'4:00-8:00':<12} | {ch2:>11.4f} | {dis2:>11.4f} |")

        ch3 = np.sum(res['ch'][48:72]); dis3 = np.sum(res['dis'][48:72])
        ch4 = np.sum(res['ch'][72:96]); dis4 = np.sum(res['dis'][72:96])
        print(f"| {'8:00-12:00':<12} | {ch3:>11.4f} | {dis3:>11.4f} | {'12:00-16:00':<12} | {ch4:>11.4f} | {dis4:>11.4f} |")

        ch5 = np.sum(res['ch'][96:120]); dis5 = np.sum(res['dis'][96:120])
        ch6 = np.sum(res['ch'][120:144]); dis6 = np.sum(res['dis'][120:144])
        print(f"| {'16:00-20:00':<12} | {ch5:>11.4f} | {dis5:>11.4f} | {'20:00-24:00':<12} | {ch6:>11.4f} | {dis6:>11.4f} |")
        print(f"+{'-'*14}+{'-'*13}+{'-'*13}+{'-'*14}+{'-'*13}+{'-'*13}+")
        print(f"| {'0:00 储电量':<12} | {res['s0']:>11.4f} | {'-':>11} | {'24:00 储电量':<12} | {res['s24']:>11.4f} | {'-':>11} |")
        print(f"+{'-'*14}+{'-'*13}+{'-'*13}+{'-'*14}+{'-'*13}+{'-'*13}+")

def display_table3(results, model_label):
    print(f"\n{'='*108}")
    print(f"                                    【{model_label}】表 3　微网在指定日期的紧急购电量")
    print(f"{'='*108}")

    day_records = {}
    max_len = 0
    for d in TYPICAL_DATES:
        em_vec = results[d]['em']
        em_idx = np.where(em_vec > 0.01)[0]
        records = []
        if len(em_idx) > 0:
            curr_s = em_idx[0]; curr_p = em_idx[0]
            for i in em_idx[1:]:
                if i == curr_p + 1:
                    curr_p = i
```

```
                else:
                    h1, m1 = divmod(curr_s * 10, 60); h2, m2 = divmod((curr_p + 1) * 10, 60)
                    t_str = f"{h1:02d}:{m1:02d}-24:00" if h2 == 24 and m2 == 0 else f"{h1:02d}:{m1:02d}-{h2:02d}:{m2:02d}"
                    records.append((t_str, np.sum(em_vec[curr_s:curr_p+1])))
                    curr_s = i; curr_p = i
            h1, m1 = divmod(curr_s * 10, 60); h2, m2 = divmod((curr_p + 1) * 10, 60)
            t_str = f"{h1:02d}:{m1:02d}-24:00" if h2 == 24 and m2 == 0 else f"{h1:02d}:{m1:02d}-{h2:02d}:{m2:02d}"
            records.append((t_str, np.sum(em_vec[curr_s:curr_p+1])))
        day_records[d] = records
        if len(records) > max_len:
            max_len = len(records)

    print(f"+{'-'*25}+{'-'*25}+{'-'*25}+{'-'*25}+")
    print(f"| {TYPICAL_DATES[0]:^23} | {TYPICAL_DATES[1]:^23} | {TYPICAL_DATES[2]:^23} | {TYPICAL_DATES[3]:^23} |")
    print(f"+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+")
    print(f"| {'时间段':^11} | {'购电量':^9} | {'时间段':^11} | {'购电量':^9} | {'时间段':^11} | {'购电量':^9} | {'时间段':^11} | {'购电量':^9} |")
    print(f"+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+")

    if max_len == 0:
        print(f"| {'全天无缺额':^11} | {0.0:>9.4f} | {'全天无缺额':^11} | {0.0:>9.4f} | {'全天无缺额':^11} | {0.0:>9.4f} | {'全天无缺额':^11} | {0.0:>9.4f} |")
    else:
        for r_idx in range(max_len):
            row_str = "|"
            for d in TYPICAL_DATES:
                recs = day_records[d]
                if r_idx < len(recs):
                    t_str, vol_val = recs[r_idx]
                    row_str += f" {t_str:^11} | {vol_val:>9.4f} |"
                else:
                    if len(recs) == 0 and r_idx == 0:
                        row_str += f" {'全天无缺额':^11} | {0.0:>9.4f} |"
                    else:
                        row_str += f" {'-':^11} | {'-':^9} |"
            print(row_str)
    print(f"+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+{'-'*13}+{'-'*11}+")

# ========================================
```

```
# 7. 导出至 result4-2.xlsx 与 result4-3.xlsx
# ========================================
def merge_continuous_emergency(em_vec, date_str):
    """
    识别紧急购电向量中的连续激活时步，将其合并为一个连续时间段区间，并累加购电量
    示例：若 78~80 连续有时步缺额，合并为 '13:00-13:30'，购电量取该区间之和
    """
    rows_em = []
    em_idx = np.where(em_vec > 0.01)[0]
    if len(em_idx) == 0:
        return rows_em

    curr_s = em_idx[0]
    curr_p = em_idx[0]
    ranges = []
    for i in em_idx[1:]:
        if i == curr_p + 1:
            curr_p = i
        else:
            ranges.append((curr_s, curr_p))
            curr_s = i
            curr_p = i
    ranges.append((curr_s, curr_p))

    for st, ed in ranges:
        h1, m1 = divmod(st * 10, 60)
        h2, m2 = divmod((ed + 1) * 10, 60)
        t_str = f"{h1:02d}:{m1:02d}-24:00" if h2 == 24 and m2 == 0 else
f"{h1:02d}:{m1:02d}-{h2:02d}:{m2:02d}"
        em_val = np.sum(em_vec[st:ed+1])
        rows_em.append({'日期': date_str, '购电时间段': t_str, '购电量': round(em_val, 4)})

    return rows_em

def export_to_excel(date_list, results_4_2, results_4_3):
    print("\n>>> 正在将完整购电策略写入 Excel 结果文件...")
    target_dates = [d for d in date_list if d >= '2025-02-01']

    # 尝试加载模板列名结构，或自动构建标准 144 步时间段列名
    interval_cols = []
    template_found = False
    if os.path.exists('result4-2.xlsx'):
```

```
        try:
            df_tpl = pd.read_excel('result4-2.xlsx', sheet_name='计划购电量', nrows=1)
            if len(df_tpl.columns) == 147:
                interval_cols = df_tpl.columns[1:145].tolist()
                template_found = True
        except Exception:
            pass

    if not template_found:
        for t in range(STEPS_PER_DAY):
            st_m = t * 10; ed_m = (t + 1) * 10
            h1, m1 = divmod(st_m, 60); h2, m2 = divmod(ed_m, 60)
            if h2 == 24 and m2 == 0:
                interval_cols.append("0:00-0:10+1")
            else:
                interval_cols.append(f"{h1}:{m1:02d}-{h2}:{m2:02d}")

    # ---------------- 写入 result4-2.xlsx ----------------
    with pd.ExcelWriter('result4-2.xlsx', engine='openpyxl') as writer:
        rows_plan = []
        for d in target_dates:
            res = results_4_2[d]
            row = [d] + list(np.round(res['plan'], 4)) + [round(res['plan_vol'], 4),
round(res['plan_cost'], 4)]
            rows_plan.append(row)
        pd.DataFrame(rows_plan, columns=['日期\\时间'] + interval_cols + ['全天购电量', '全天
购电费']
                     ).to_excel(writer, sheet_name='计划购电量', index=False)

        rows_soc = []
        for d in target_dates:
            res = results_4_2[d]
            for b_idx, (b_name, st, ed) in enumerate(TABLE2_BLOCKS):
                ch_val = np.sum(res['ch'][st:ed])
                dis_val = np.sum(res['dis'][st:ed])
                if b_idx == 0:
                    t_mark, s_val = '00:00:00', round(res['s0'], 4)
                elif b_idx == 1:
                    t_mark, s_val = '24:00:00', round(res['s24'], 4)
                else:
                    t_mark, s_val = np.nan, np.nan
                rows_soc.append({
```

```
                    '日期': d, '时间段': b_name, '充电量': round(ch_val, 4),
                    '放电量': round(dis_val, 4), '时刻': t_mark, '储电量': s_val
                })
        pd.DataFrame(rows_soc).to_excel(writer, sheet_name='充放电量', index=False)

        # 紧急购电量：连续时间段合并
        rows_em = []
        for d in target_dates:
            rows_em.extend(merge_continuous_emergency(results_4_2[d]['em'], d))
        if len(rows_em) == 0:
            rows_em.append({'日期': np.nan, '购电时间段': np.nan, '购电量': np.nan})
        pd.DataFrame(rows_em).to_excel(writer, sheet_name='紧急购电量', index=False)

    print(" -> [成功保存] result4-2.xlsx (紧急购电连续时间段已合并)")

    # ---------------- 写入 result4-3.xlsx ----------------
    with pd.ExcelWriter('result4-3.xlsx', engine='openpyxl') as writer:
        rows_plan3 = []
        for d in target_dates:
            res = results_4_3[d]
            row = [d] + list(np.round(res['plan'], 4)) + [round(res['plan_vol'], 4),
round(res['plan_cost'], 4)]
            rows_plan3.append(row)
        pd.DataFrame(rows_plan3, columns=['日期\\时间'] + interval_cols + ['全天购电量', '全天
购电费']
                     ).to_excel(writer, sheet_name='计划购电量', index=False)

        rows_adj = []
        for d in target_dates:
            res = results_4_3[d]
            row = [d] + list(np.round(res['adj'], 4)) + [round(res['exec_vol'], 4), round(res['cost'],
4)]
            rows_adj.append(row)
        pd.DataFrame(rows_adj, columns=['日期\\时间'] + interval_cols + ['全天购电量', '全天购
电费']
                     ).to_excel(writer, sheet_name='调整购电量', index=False)

        rows_soc3 = []
        for d in target_dates:
            res = results_4_3[d]
            for b_idx, (b_name, st, ed) in enumerate(TABLE2_BLOCKS):
                ch_val = np.sum(res['ch'][st:ed])
```

```
                dis_val = np.sum(res['dis'][st:ed])
                if b_idx == 0:
                    t_mark, s_val = '00:00:00', round(res['s0'], 4)
                elif b_idx == 1:
                    t_mark, s_val = '24:00:00', round(res['s24'], 4)
                else:
                    t_mark, s_val = np.nan, np.nan
                rows_soc3.append({
                    '日期': d, '时间段': b_name, '充电量': round(ch_val, 4),
                    '放电量': round(dis_val, 4), '时刻': t_mark, '储电量': s_val
                })
        pd.DataFrame(rows_soc3).to_excel(writer, sheet_name='充放电量', index=False)

        # 紧急购电量：连续时间段合并
        rows_em3 = []
        for d in target_dates:
            rows_em3.extend(merge_continuous_emergency(results_4_3[d]['em'], d))
        if len(rows_em3) == 0:
            rows_em3.append({'日期': np.nan, '购电时间段': np.nan, '购电量': np.nan})
        pd.DataFrame(rows_em3).to_excel(writer, sheet_name='紧急购电量', index=False)

    print(" -> [成功保存] result4-3.xlsx (紧急购电连续时间段已合并)")

# ========================================
# 8. 主流程执行入口
# ========================================
if __name__ == '__main__':
    # 1. 载入输入数据并进行 PCHIP 插值
    date_list, P_load_dict, P_pv_dict, Price_dict, PV_forecast_dict = load_data()

    # 2. 全年求解计算
    results_4_2, results_4_3 = run_all_models(
        date_list, P_load_dict, P_pv_dict, Price_dict, PV_forecast_dict
    )

    # 3. 严格输出【波动电价下问题 2】的表 1、表 2、表 3
    print("\n" + "#" * 108)
    print("                                【一、波动电价下 - 问题 2 的全部报表展示 (日前鲁棒策略)】
")
    print("#" * 108)
    display_table1(results_4_2, model_label="问题 4-2", is_q3=False)
    display_table2(results_4_2, model_label="问题 4-2")
```

```
    display_table3(results_4_2, model_label="问题 4-2")

    # 4. 严格输出【波动电价下问题 3】的表 1、表 2、表 3
    print("\n" + "#" * 108)
    print("                                【二、波动电价下 - 问题 3 的全部报表展示 (日内滚动 MPC 策略)】")
    print("#" * 108)
    display_table1(results_4_3, model_label="问题 4-3", is_q3=True)
    display_table2(results_4_3, model_label="问题 4-3")
    display_table3(results_4_3, model_label="问题 4-3")

    # 5. 年度总体对比与论文加分指标汇总
    cost_tot_42 = sum(results_4_2[d]['cost'] for d in date_list if d >= '2025-02-01')
    cost_tot_43 = sum(results_4_3[d]['cost'] for d in date_list if d >= '2025-02-01')
    em_tot_42 = sum(np.sum(results_4_2[d]['em']) for d in date_list if d >= '2025-02-01')
    em_tot_43 = sum(np.sum(results_4_3[d]['em']) for d in date_list if d >= '2025-02-01')

    print("\n" + "=" * 92)
    print("                【三、波动电价下 2025.2.1 - 2025.12.31 综合运行指标与降本效益对比】")
    print("=" * 92)
    print(f" 方案名称                    | 全年累计总购电费用 (元) | 全年紧急购电量 (kWh) | 紧急购电总时段数")
    print("-" * 92)
    print(f" 问题 4-2 (日前鲁棒)          | {cost_tot_42:>22.2f} | {em_tot_42:>20.2f} | {sum(len(np.where(results_4_2[d]['em']>0.01)[0]) for d in date_list if d >= '2025-02-01'):>16}")
    print(f" 问题 4-3 (滚动 MPC)          | {cost_tot_43:>22.2f} | {em_tot_43:>20.2f} | {sum(len(np.where(results_4_3[d]['em']>0.01)[0]) for d in date_list if d >= '2025-02-01'):>16}")
    savings = cost_tot_42 - cost_tot_43
    ratio = (savings / cost_tot_42) * 100 if cost_tot_42 > 0 else 0.0
    print("-" * 92)
    print(f" ★ 滚动 MPC 调控增益      : 累计为微网节省购电成本 {savings:,.2f} 元 (综合降本率: {ratio:.2f}%)")
    print("=" * 92)

    # 6. 保存输出标准 Excel 文件
    export_to_excel(date_list, results_4_2, results_4_3)
    print("\n>>> 全部计算、报表渲染与文件导出已圆满执行完成！")
```